

UNIVERSITÉ JOSEPH KI-ZERBO

(UJKZ)

INSTITUT BURKINABE DES ARTS ET MÉTIERS

(IBAM)



**RAPPORT DE STAGE POUR L'OBTENTION DE LA LICENCE
PROFESSIONNELLE**

OPTION : Méthodes Informatiques Appliquées à la Gestion (MIAGE)

Période de stage : du 08 Juillet au 08 Octobre 2025

THÈME :
**CONCEPTION ET RÉALISATION D'UN SYSTEME DIGITAL
DE SUIVI DES FACTURES ET DES COMMANDES DANS
UNE PME : CAS DE ESOF BURKINA SARL.**

Présenté par **Abdoul Razack OUEDRAOGO**

Maitre de stage :

M. Vivian BOGNINI

Directeur Général de

ESOF BURKINA

Directeur de rapport :

Dr Yaya TRAORE

Maître de conférences en

Informatique / IBAM-UJKZ

Année académique 2024-2025

SOMMAIRE

SOMMAIRE	i
DÉDICACE	ii
REMERCIEMENTS	iii
LISTE DES SIGLES ET ABREVIATIONS	iv
LISTE DES FIGURES ET GRAPHIQUES	vi
LISTE DES TABLEAUX	vii
INTRODUCTION GÉNÉRALE	1
CHAPITRE I : Présentation des structures de formations et d'accueil	2
I. Présentation de la structure de formation	2
II. Présentation de la structure d'accueil.....	5
CHAPITRE II : Analyse et la conception	9
I. Étude préalable	9
II. Expression des besoins.....	17
III. Conception globale.....	37
IV. Réalisation	43
CHAPITRE III : Bilan du stage	66
I. Déroulement du stage et les activités menées	66
II. Observations et suggestions	67
CONCLUSION GÉNÉRALE	68
BIBLIOGRAPHIE ET WEBOGRAPHIE	I
Table des matières	IV
ANNEXES	VII
I. ANNEXE 1 : Langage UML.....	VII
II. ANNEXE 2 : Le dictionnaires des données.	VII

DÉDICACE

A

toutø notrø

famillø

REMERCIEMENTS

La réalisation de ce rapport de stage et l'aboutissement de cette expérience professionnelle n'auraient pu être possibles sans le soutien et l'accompagnement de nombreuses personnes, auxquelles nous tenons à exprimer notre profonde gratitude.

Nous adressons tout particulièrement nos sincères remerciements :

- à **Dr Yaya TRAORÉ**, notre professeur de suivi, pour son encadrement, ses conseils et sa disponibilité. Ses orientations méthodologiques ont été déterminantes dans l'élaboration de ce rapport ;
- à **M. Vivian BOGNINI** Directeur Général de ESOFT BURKINA et notre maître de stage, pour nous avoir accueillis au sein de son entreprise, pour son accompagnement personnel, ses orientations et son écoute bienveillante qui ont grandement facilité notre intégration et notre apprentissage ;
- à l'ensemble du personnel d'ESOFT BURKINA pour leur accueil chaleureux, leur encadrement et le généreux partage de leur expertise, qui ont enrichi nos compétences techniques et relationnelles ;
- au **Pr Gilbert BAYILI**, Directeur actuel de l'IBAM, ainsi que tout le corps professoral pour la qualité de la formation et les solides bases académiques qui ont enrichi notre compréhension tout au long de ce stage ;
- à nos parents, famille, amis et camarades de promotion pour leur soutien constant, leurs encouragements et leur présence dans les moments de questionnement. Leur appui a été une source précieuse de motivation ;
- enfin, à toutes les personnes qui, de près ou de loin, ont contribué à la réussite de ce travail, trouvent ici l'expression de notre profonde reconnaissance.

LISTE DES SIGLES ET ABBREVIATIONS

SIGLE / ABBRÉVIATION	SIGNIFICATION
2TUP	Two Tracks Unified Process
ABF	Assurance Banque Finance
ADB	Assistant de Direction/Bilingue
ADC	Assistant de Direction/Comptable
ANPFI	Agence nationale de promotion de la finance inclusive
API	Application Programming Interface
ATOS	Administratifs, Techniciens, Ouvriers et Services.
BDD	Base de données
CCA	Comptabilité Contrôle Audit
CCI	Chambre de Commerce et d'Industrie
COCOMO	COConstructive COst Model
CSAF	Chef de Service Administratif, Financier
CU	Cas d'Utilisation
GBE	Garantie de bonne exécution
IBAM	Institut Burkinabè des Arts et Métiers
IDE	Integrated Development Environment
IHM	Interface Homme–Machine
JSON	JavaScript Object Notation
JSON/RPC	<i>JSON Remote Procedure Call</i>
JWT	JSON Web Token
MAGE	Master en Administration et Gestion des Entreprises
MBF	Master en Banque-Finance

MEF	Ministère de l'Economie et des Finances
MCCA	Master en Comptabilité-Contrôle-Audit
MG	Marketing et Gestion
MID	Marketing et Innovation Digitale
MIAGE	Méthodes Informatiques Appliquées à la Gestion
M2ISIE	Master en Informatique, option Ingénierie des Systèmes d'Informations en Entreprise
PME	Petite et Moyenne Entreprise
RUP	Rational Unified Process
SGBD	Système de Gestion de Base de Données
SI	Système d'Information
SQL	Structured Query Language
SSL	Secure Sockets Layer
SARL	Société à Responsabilité Limitée
UML	Unified Modeling Language
HTTPS	HyperText Transfer Protocol Secure
HTML	HyperText Markup Language
XP	eXtreme Programming

LISTE DES FIGURES ET GRAPHIQUES

Figure 1: Organigramme de l'IBAM.	3
Figure 2 : Organigramme de ESOFT BURKINA	8
Figure 3 : Schéma illustratif du processus 2TUP	13
Figure 4 : Planning prévisionnel du projet	17
Figure 5 : Processus de commande et de facturation (Vue Client / ESOFT).....	18
Figure 6 : Processus de commande et de facturation (Vue ESOFT / Fournisseur).....	19
Figure 7: Diagramme de cas d'utilisation	27
Figure 8 : Diagramme de cas de séquence « S'authentifier »	32
Figure 9 : Diagramme de séquence « Enregistrer une facture client ».....	33
Figure 10 : Diagramme de cas de séquence « Associer une garantie de bonne exécution » ...	34
Figure 11 : Diagramme de séquence « Enregistrer un paiement client ».....	35
Figure 12 : Architecture 3-tiers : Schéma illustratif.....	37
Figure 13 : Le diagramme des classes.....	40
Figure 14 : Diagramme d'état-transition de l'objet FactureClient	41
Figure 15 : Diagramme d'état-transition d'un objet GarantieBonneExecution	42
Figure 16 : Diagramme de déploiement	43
Figure 17: Schéma d'illustration de l'architecture MVC	51
Figure 18 : Écran page de destination	52
Figure 19 :Écran de connexion.....	52
Figure 20 : Écran d'enregistrement d'une facture client.....	53
Figure 21 : Écran de quelques factures clients enregistrées.....	53
Figure 22 : Rapport de quelques tests cas de test unitaires	57
Figure 23 : : Rapport de quelques tests cas de test fonctionnels	58

LISTE DES TABLEAUX

<i>Tableau 1 : Comparaison des processus ou cycles de développement</i>	11
Tableau 2 : Comparaison de quelques langages de modélisation	15
Tableau 3 : Évaluation comparative d’autres logiciels de facturation	22
<i>Tableau 4: Listes des cas d'utilisation</i>	24
Tableau 5: Description textuelle du cas d'utilisation « S'authentifier »	28
Tableau 6 : : Description textuelle du cas d'utilisation « Enregistrer une facture client »	29
<i>Tableau 7 : Description textuelle du cas d'utilisation « Associer une garantie de bonne exécution »</i>	30
Tableau 8 : Description textuelle du cas d'utilisation « Enregistrer un paiement client ».	30
<i>Tableau 9 : Comparaison des différentes architectures de développement</i>	36
Tableau 10 : Dictionnaire de données de la classe « Structure »	38
Tableau 11 : Dictionnaire de données de la classe « Facture »	38
Tableau 12 : Dictionnaire de données de la classe « ReglementFacture »	38
Tableau 13 : Dictionnaire de données de la classe « Retenue »	39
Tableau 14 : Dictionnaire de données de la classe « GarantieBonneExecution »	39
Tableau 15 : Comparaison de quelques frameworks modernes	45
Tableau 16 : Comparaison de quelques frameworks frontend populaires	45
Tableau 17 : Comparaison de différents serveurs web	47
Tableau 18 : Comparaison de quelques systèmes de gestion de bases de données relationnels	49
Tableau 19 : Jeu de test unitaire avant le test	55
Tableau 20 : Jeu de test fonctionnel avant le test	56
Tableau 21 : Jeu de test unitaire après le test	58
Tableau 22: Jeu de test fonctionnel après le test	58
Tableau 23 : Comparaison des méthodes d'estimation de coût	61
Tableau 24 : Estimation du cout de développement	63
Tableau 25 : Coût global de réalisation du projet	64
Tableau 26 : Dictionnaire de données de la classe « OdooConfiguration »	VIII
Tableau 27 : Dictionnaire de données de la classe « Structure »	VIII
Tableau 28 : Dictionnaire de données de la classe « TypeStructure »	VIII
Tableau 29 : Dictionnaire de données de la classe « Domaine »	IX
Tableau 30 : Dictionnaire de données de la classe « Contact »	IX
Tableau 31 : Dictionnaire de données de la classe « Compte »	IX
Tableau 32 : Dictionnaire de données de la classe « TypeCompte »	IX
Tableau 33 : Dictionnaire de données de la classe « Facture »	X
Tableau 34 : Dictionnaire de données de la classe « ReglementFacture »	X
Tableau 35 : Dictionnaire de données de la classe « ModePaiement »	X
Tableau 36 : Dictionnaire de données de la classe « Retenue »	XI
Tableau 37 : Dictionnaire de données de la classe « GarantieBonneExecution »	XI

Tableau 38 : Dictionnaire de données de la classe « Banque ».....	XI
Tableau 39 : Dictionnaire de données de la classe « Utilisateur »	XII
Tableau 40 : Dictionnaire de données de la classe « Role »	XII
Tableau 41 : Dictionnaire de données de la classe « Permission »	XII

INTRODUCTION GÉNÉRALE

La transformation numérique constitue aujourd'hui un levier essentiel pour optimiser le fonctionnement interne des entreprises et renforcer leur compétitivité. Pour les petites et moyennes entreprises (PME), l'enjeu principal réside dans la modernisation de leurs procédures administratives et financières, afin d'améliorer la productivité, de limiter les erreurs et de réduire les pertes liées aux oublis.

C'est dans cette dynamique que l'entreprise ESOFT BURKINA a engagé une réflexion sur l'optimisation du suivi de ses factures clients et fournisseurs. Le dispositif actuel, encore semi-manuel, présente en effet des limites notables en matière de traçabilité, de régularité des règlements et de gestion des retenues et garanties de bonne exécution, générant retards et risques financiers.

Dans ce contexte, nous avons effectué un stage de trois mois au sein de ladite entreprise, avec pour mission de concevoir et de mettre en œuvre une plateforme numérique répondant à ces besoins spécifiques, en nous appuyant sur une méthodologie adaptée et sur des technologies modernes. Ce rapport présente le fruit de ce travail et s'articule en trois chapitres : le premier introduit le cadre académique et professionnel du stage, le deuxième expose l'analyse et la conception du système proposé, et le troisième dresse le bilan du stage en mettant en évidence les acquis, les difficultés rencontrées et les perspectives d'évolution.

CHAPITRE I : Présentation des structures de formations et d'accueil

Ce premier chapitre est consacré à la présentation de notre structure de formation, l'Institut Burkinabè des Arts et Métiers (IBAM), ainsi qu'à celle de notre structure d'accueil, ESOFT BURKINA SARL. L'objectif est de définir le cadre académique et professionnel dans lequel s'inscrit notre projet de stage.

I. Présentation de la structure de formation

Situé à Somgandé, l'IBAM est un institut d'enseignement professionnel relevant de l'Université Joseph KI-ZERBO. Dans cette section, nous présentons une vue d'ensemble de l'institut en mettant en évidence son historique, ses objectifs, son organisation institutionnelle ainsi que ses principales offres de formation.

1. Historique

L'IBAM a été créé en janvier 2000, à l'occasion de la refondation de l'Université de Ouagadougou, rebaptisée « Université Ouaga I Pr Joseph KI-ZERBO » le 26 décembre 2015, puis « Université Joseph KI-ZERBO » le 12 avril 2019. L'IBAM constitue la concrétisation de l'engagement de l'Université Joseph KI-ZERBO (UJKZ) en faveur de la professionnalisation des filières de formation, dans le but de renforcer l'efficacité externe de l'Université.

2. Objectif

La mission principale de l'IBAM est de former des cadres moyens et supérieurs compétents, aptes à répondre efficacement aux exigences du marché du travail. Cette orientation vise à faciliter l'insertion professionnelle des diplômés et s'aligne sur les missions fondamentales de l'Université Joseph KI-ZERBO, qui œuvre pour le développement socio-économique du Burkina Faso.

3. Organisation

L'IBAM est organisé en deux organes, à savoir l'organe statutaire et l'organe d'exécution. La répartition des enseignants ainsi que les modalités de contrôle des aptitudes et des connaissances sont fixées par arrêté ministériel. L'Institut est placé sous la direction d'un Directeur, assisté d'un Directeur adjoint, tous deux élus par le collège électoral de l'établissement pour un mandat de deux (02) ans renouvelables.

La *Figure 1* présente l'organisation détaillée de l'IBAM.

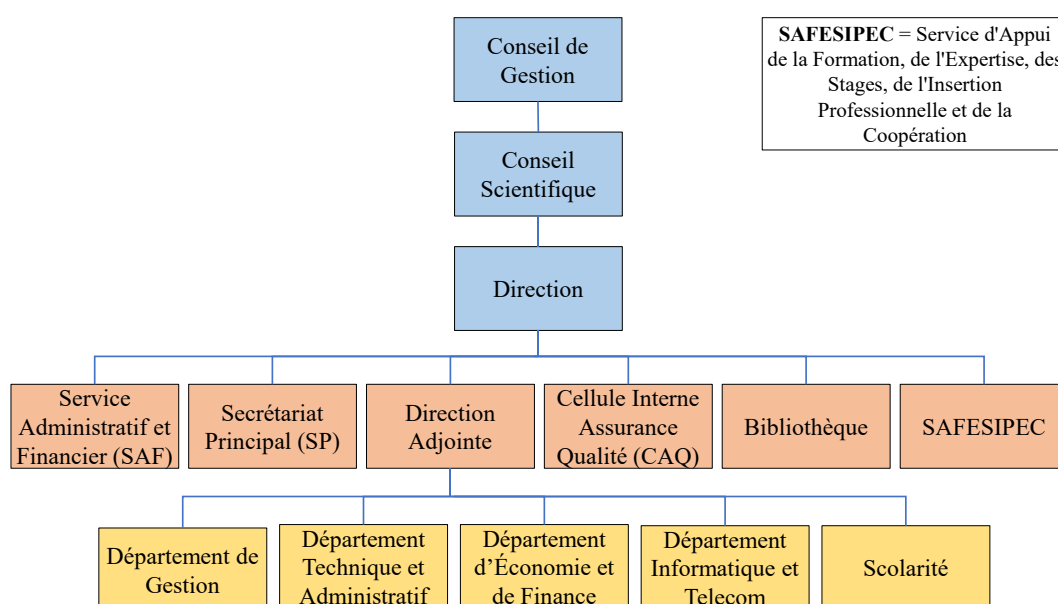


Figure 1: Organigramme de l'IBAM.

a). L'organe statutaire

L'organe statutaire de l'IBAM est constitué du Conseil de gestion et du Conseil scientifique :

- Le **Conseil de gestion** constitue l'organe décisionnel de l'Institut. Il est composé du Directeur de l'IBAM, du Directeur adjoint, des Coordonnateurs de sections, du Secrétaire principal, du chef du service administratif et financier CSAF, des Enseignants permanents, de deux (02) représentants des Délégués de syndicats des étudiants et d'un représentant du personnel ATOS. Il est présidé par le Directeur de l'IBAM et se réunit tous les trois (03) mois, sauf en cas de session extraordinaire convoquée par son président. Le conseil de gestion a pour mission de :
 - définir le règlement intérieur de l'institut ;
 - proposer le budget de fonctionnement de l'institut ;
 - décider des mesures administratives et financières conformément aux textes en vigueur ;
 - proposer la modification des statuts de l'IBAM.
- Le **Conseil scientifique** est composé également du Directeur de l'IBAM, du Directeur adjoint, des Coordonnateurs de section, des enseignants de rang A et les enseignants de rang B de l'Institut. Il est présidé par le Directeur de l'IBAM. Il se réunit et délibère

dans les mêmes conditions que le Conseil de gestion mais son secrétariat est assuré par un des Coordonnateurs de section. Le Conseil scientifique a pour mission de :

- proposer une organisation générale des enseignements de l’institut ;
- proposer l’ouverture, la fusion ou la fermeture de filières ;
- créer des titres, diplômes et d’examiner les équivalences pédagogiques ;
- proposer la création de nouveaux postes et d’apprécier les orientations de la recherche au sein de l’institut.

b). L’organe d’exécution

L’organe d’exécution a pour rôle d’appliquer les décisions prises par l’organe statutaire. Il est essentiellement constitué de la Direction de l’IBAM. La Direction est placée sous la responsabilité d’un Directeur, chargé de diriger et de superviser le fonctionnement des différents services, ainsi que de définir la politique générale de l’Institut. Elle coordonne les activités des différents services et veille au maintien de l’image de marque de l’Institut. En outre, elle veille à la mise en œuvre des décisions, des projets et des programmes de l’Institut. Par ailleurs, elle veille à la mise en application des textes et règlements de l’établissement.

Le Directeur de l’IBAM est assisté par une Secrétaire de direction. Rappelons que le Secrétaire principal et le CSAF assistent également le Directeur dans l’accomplissement de certaines activités en lien avec leurs domaines de spécialisation. La Direction est composée du Secrétariat de direction, du Service administratif et financier, et du Secrétariat principal.

4. Filières de formation

En vue d’atteindre les objectifs cités précédemment, l’IBAM offre des formations dans plusieurs filières. Ces formations sont organisées en deux (02) groupes selon le diplôme préparé : les licences professionnelles et les masters.

a). Licences professionnelles

Les principales filières de licence professionnelle sont :

- Assurance Banque Finance (ABF) ;
- Assistant de Direction/Bilingue (ADB) ;
- Assistant de Direction Comptable (ADC) ;
- Comptabilité-Contrôle-Audit (CCA) ;
- Informatique, option Méthodes Informatiques Appliquées à la Gestion (MIAGE) ;
- Informatique, option Réseaux et Télécommunications (RI-MT) ;

- Marketing et Innovation Digitale (MID).

Il convient de noter que, depuis l'année académique 2021-2022, l'IBAM a arrêté la filière Marketing et Gestion (MG). La filière Marketing et Innovation Digitale (MID) a été introduite en 2022-2023 en remplacement et en réadaptation de l'ancienne formation.

b). Masters

Au niveau masters, l'IBAM offre cinq (05) filières de formations qui sont :

- Master en Administration et Gestion des Entreprises (MAGE) ;
- Master en Banque-Finance (MBF) ;
- Master en Comptabilité-Contrôle-Audit (MCCA) ;
- Master en Informatique, option Ingénierie des Systèmes d'Informations en Entreprise (M2ISIE) ;
- Master en Informatique, option Sécurité Informatique (MISI).

La filière MAGE a été créée pour répondre aux besoins des entreprises en cadres qualifiés dans le domaine des technologies de l'information. Elle est ouverte en première année aux titulaires du baccalauréat des séries C, D et E ayant réussi au test de sélection ou admis comme auditeurs libres. En fin de cycle, les étudiants doivent effectuer un stage pratique d'une durée minimale de trois (03) mois, sanctionné par une soutenance publique. Ce stage constitue une opportunité pour se familiariser avec le milieu professionnel et mettre en pratique les connaissances acquises. C'est dans ce cadre que nous avons été accueillis au sein de l'entreprise ESOF BURKINA.

II. Présentation de la structure d'accueil

Cette partie est consacrée à la présentation de notre structure d'accueil, **ESOF BURKINA**, à travers son identité, son historique, ses missions, ses domaines de compétence, ses produits, ses partenaires ainsi que son organisation interne.

1. Identité

ESOF BURKINA SARL est une Société à Responsabilité Limitée (SARL), dynamique et innovante, basée à Saaba (Ouagadougou). Évoluant dans les secteurs de l'informatique, des télécommunications, de l'énergie et du BTP/Mining, elle se distingue par son expertise à fournir des solutions sur mesure pour accompagner la transformation numérique et énergétique des acteurs publics et privés.

2. Historique

Fondée en juillet 2016, ESOFT BURKINA s'est rapidement imposée comme un acteur de référence dans l'intégration de solutions numériques et énergétiques au Burkina Faso. Dès ses débuts, l'entreprise a mis l'accent sur l'innovation, la qualité et l'efficacité opérationnelle, ce qui lui a permis de gagner la confiance de nombreux partenaires. Au fil des années, elle a élargi son champ d'intervention et consolidé sa notoriété à travers des projets d'envergure et des collaborations stratégiques avec des institutions, des entreprises privées et des organisations internationales.

3. Mission et objectifs stratégiques

ESOFT BURKINA poursuit plusieurs objectifs stratégiques qui guident ses actions, notamment :

- répondre efficacement aux besoins de ses clients dans les meilleures conditions ;
- fournir des services innovants et des produits de haute qualité ;
- optimiser et sécuriser les infrastructures informatiques et énergétiques des organisations;
- développer et fidéliser une clientèle diversifiée au niveau national et international ;
- renforcer son positionnement comme un acteur incontournable de la digitalisation au Burkina Faso.

4. Domaines de compétence et prestations

Les activités de l'entreprise s'articulent autour de trois grands pôles :

- **Informatique et Télécommunications**
 - développement web et mobile ;
 - gestion et suivi de projets informatiques ;
 - sécurité informatique et contrôle d'accès ;
 - construction et intégration de systèmes de télécommunications ;
 - vente et maintenance de matériels informatiques et télécoms.
- **Énergie et Solaire**
 - installation de systèmes solaires photovoltaïques ;
 - électricité industrielle et de bâtiment ;
 - réseaux électriques HTA/BT ;
 - distribution et vente de matériels électriques et solaires.
- **BTP et Mining**

- conception et réalisation de bâtiments et ouvrages d'art ;
- travaux routiers et entretien d'infrastructures ;
- installation et maintenance de systèmes de froid et climatisation ;
- construction, fourniture et réparation d'équipements de Mining.

5. Produits digitaux et partenaires

ESOFT BURKINA est à l'origine de plusieurs projets numériques innovants ayant un impact significatif au Burkina Faso. Parmi les plus notables, on distingue :

- **maformation.bf**, une plateforme de formation en ligne et en présentiel, avec notamment un service de préparation aux concours administratifs ;
- **aumarche.biz**, une plateforme de commerce en ligne ouverte au grand public ;
- **warikalan.bf**, une plateforme d'éducation financière soutenue par le ministère de l'Économie et des Finances, destinée à promouvoir l'inclusion financière.

L'entreprise a également développé des partenariats stratégiques avec divers acteurs institutionnels, privés et internationaux. Parmi ses principaux partenaires, figurent : le Ministère de l'Économie et des Finances ;

- la SONABEL ;
- l'institut International d'Ingénierie de l'Eau et de l'Environnement (2iE) ;
- des organismes internationaux tels que l'UNFPA ;
- des entreprises privées comme Endeavour Mining ;
- ainsi que plusieurs ONG locales et internationales, qui bénéficient de solutions numériques adaptées à leurs besoins.

6. Organisation

ESOFT BURKINA repose sur une équipe pluridisciplinaire d'environ une trentaine de consultants et experts, regroupés autour de pôles de compétences complémentaires :

- **consultants – Architectes techniques** : chargés de définir et d'optimiser les architectures techniques des systèmes d'information, d'assurer leur sécurité et leur performance, et de piloter les projets d'intégration.
- **consultants en Assistance à Maîtrise d'Ouvrage (AMO)** : spécialisés dans l'expression des besoins, la rédaction de cahiers des charges, les études de faisabilité et le suivi fonctionnel des projets.

- **consultants en Gestion de projet** : responsables de la planification, du pilotage et du respect des contraintes de coûts, délais et qualité.

Cette organisation flexible et complémentaire permet à l'entreprise d'offrir un accompagnement global et sur mesure à ses clients.

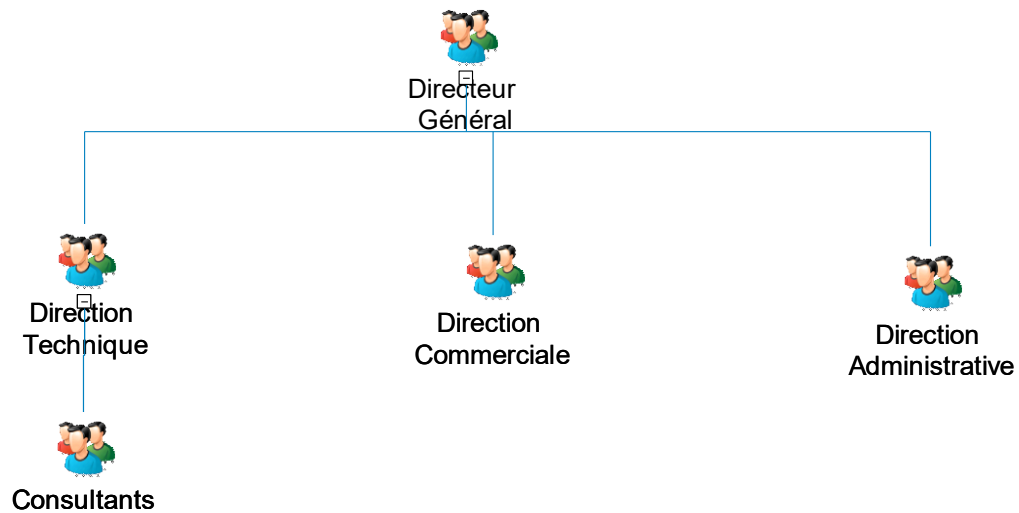


Figure 2 : Organigramme de ESOFT BURKINA

En somme, ce chapitre nous a permis de présenter notre structure de formation ainsi que notre structure d'accueil, en mettant en lumière leur identité, leurs missions et leurs domaines de compétence. Dans le chapitre suivant, nous présentons l'analyse et la conception du projet qui constituent le cœur de notre travail.

CHAPITRE II : Analyse et la conception

Ce chapitre est consacré à l'analyse et à la conception de notre projet. Il comprend l'étude préalable, l'expression des besoins, la conception globale de la solution ainsi que les étapes de sa réalisation.

I. Étude préalable

Cette section vise à fournir une compréhension approfondie du thème, de la méthodologie d'analyse et de conception, du groupe de travail et du planning de réalisation.

1. Présentation du thème

La présentation de notre thème s'articule autour de trois axes principaux, qui sont : la problématique, les objectifs et les résultats attendus.

a). Problématique

La gestion des commandes et des factures constitue un enjeu majeur pour toute entreprise de prestation de biens et de services, car elle conditionne la traçabilité des opérations et la maîtrise de la trésorerie. Chez ESOFTE BURKINA, cet enjeu est particulièrement important, car il a un impact direct sur le bon fonctionnement et la situation financière de l'entreprise.

L'entreprise dispose déjà d'un outil permettant de gérer certaines opérations telles que les ventes, les achats, l'édition de devis, de proforma, de bons de livraison, de bons de commande et de factures définitives. Toutefois, le suivi global des factures demeure complexe, notamment pour connaître leur état réel et vérifier les paiements. En pratique, certaines informations continuent d'être regroupées manuellement à l'aide de fichiers Microsoft Excel, ce qui engendre des erreurs, un manque de traçabilité et, à terme, des risques financiers.

Plus précisément, les principaux problèmes rencontrés sont :

- une mauvaise évaluation du solde réel de certaines factures, due aux paiements partiels ou multiples et aux retenues à la source mal prises en compte ;
- des oublis fréquents dans la récupération des montants liés aux garanties de bonne exécution, faute de système d'alerte ou de suivi centralisé, entraînant des pertes financières ;
- des décisions de trésorerie sous-optimales, telles que la mauvaise priorisation des paiements aux fournisseurs ou les retards dans les relances auprès des clients.

Ces constats soulignent la nécessité de repenser le dispositif actuel de suivi des factures afin de le rendre plus fiable et efficace. Dès lors, une question centrale se pose : Comment améliorer la traçabilité et le suivi des paiements des factures clients et fournisseurs au sein d'ESOFT BURKINA?

Pour répondre à cette problématique, il nous a été confié le thème : « *Conception et réalisation d'un système digital de suivi des factures et des commandes dans une PME : Cas de ESOFT BURKINA SARL* ».

b). Objectifs

Les objectifs de notre projet s'articulent autour d'une ambition principale, qui est d'améliorer la traçabilité et la maîtrise des opérations financières liées à la facturation au sein de ESOFT BURKINA.

Pour atteindre cette finalité, le projet poursuit plusieurs objectifs spécifiques, parmi lesquels :

- étudier le processus actuel de gestion des factures afin d'identifier ses limites et les besoins réels de l'entreprise ;
- concevoir et mettre en place une solution numérique adaptée ;
- mettre en application les compétences acquises durant la formation en MIAE à travers un projet concret et professionnel.

c). Résultats attendus

À l'issue de ce projet, les résultats attendus sont :

- une analyse détaillée et une conception structurée des besoins de l'entreprise ;
- une solution fonctionnelle répondant aux réalités de l'entreprise ;
- un rapport de stage rédigé et soutenu devant un jury.

2. Méthodologie

Pour assurer la réussite de notre projet, il est indispensable de suivre une démarche rigoureuse d'analyse et de conception. Celle-ci a visé à formaliser les étapes préliminaires du développement, afin de garantir une meilleure adéquation avec les besoins. La phase d'analyse permet de récolter les exigences et d'identifier les fonctionnalités attendues, tandis que la phase de conception consiste à modéliser le fonctionnement du futur système.

Dans cette optique, notre méthodologie s'appuie sur deux éléments complémentaires : le cycle de développement, qui structure notre démarche, et le langage de modélisation, qui sert à décrire et à formaliser la conception du système futur.

a). Cycle de développement

Dans le génie logiciel, la méthodologie de développement définit l'ensemble des principes et pratiques qui orientent la conduite d'un projet. Elle se concrétise à travers un cycle de développement, c'est-à-dire la manière dont les différentes phases s'enchaînent et s'organisent dans le temps.

On distingue généralement trois grandes catégories d'approches :

- les approches séquentielles, qui suivent un déroulement linéaire des phases ;
- les approches agiles, qui privilégient l'itération, l'adaptabilité et la collaboration continue ;
- les approches hybrides, qui combinent la rigueur d'une démarche structurée et la flexibilité des méthodes agiles.

Afin de retenir le cycle le plus adapté à notre projet, nous procédons à une étude comparative des principaux cycles de développement logiciel. Le *Tableau 1* ci-dessous, inspiré de travaux existants et de rapports antérieurs en présente une synthèse.

Tableau 1 : Comparaison des processus ou cycles de développement

Cycles de développement	Description	Atouts	Insuffisances
RUP¹ (Rational Unified Process)	Méthodologie itérative et incrémentale, structurée en quatre phases : Inception, Élaboration, Construction, Transition.	• Flexible et itératif • Bonne gestion des risques • Documentation complète	• Mise en œuvre complexe • Exige beaucoup de ressources • Difficile à adapter aux petites équipes
2TUP² (Two-Track Unified Process)	Approche en « Y » séparant besoins fonctionnels et aspects techniques, puis intégration progressive.	• Bon équilibre entre métier et technique • Itératif et adaptable • Bonne gestion des risques • Définition claire des rôles et livrables	• Nécessite une forte discipline et communication • Peu de modèles de documents standards

¹ RUP : https://en.wikipedia.org/wiki/Rational_Unified_Process

² 2TUP : https://fr.wikipedia.org/wiki/Two_Tracks_Unified_Process

XP³ (eXtreme Programming)	Méthode agile basée sur des pratiques comme les petits incréments, les tests automatisés et la programmation en binôme.	• Haute qualité logicielle • Forte réactivité aux changements • Encourage l'innovation grâce à la programmation en binôme	• Manque d'analyse initiale • Risque de "faire et défaire" • Peu adapté aux projets de grande envergure
Scrum⁴	Méthode agile centrée sur des itérations courtes appelées sprints, avec auto-organisation des équipes.	• Livrables fréquents et testés • Responsabilisation de l'équipe • Forte adaptabilité	• Faible documentation • Dépend d'une bonne gestion d'équipe • Peut devenir chaotique sans encadrement

L'analyse du tableau montre que chaque méthodologie possède des atouts mais également des insuffisances non négligeables, dans notre contexte :

Notre choix se porte donc sur le processus 2TUP pour sa capacité à concilier rigueur analytique et flexibilité de conception, tout en alignant nos travaux avec les attentes académiques et professionnelles, notamment en matière de production d'un rapport détaillé d'analyse et de conception. La *Figure 3* présente le schéma en « Y » qui caractérise le processus 2TUP.

³ XP : <https://www.projectmanagement.com/wikis/609078/extreme-programming>

⁴ SCRUM : <https://scrumguides.org/scrum-guide.html>

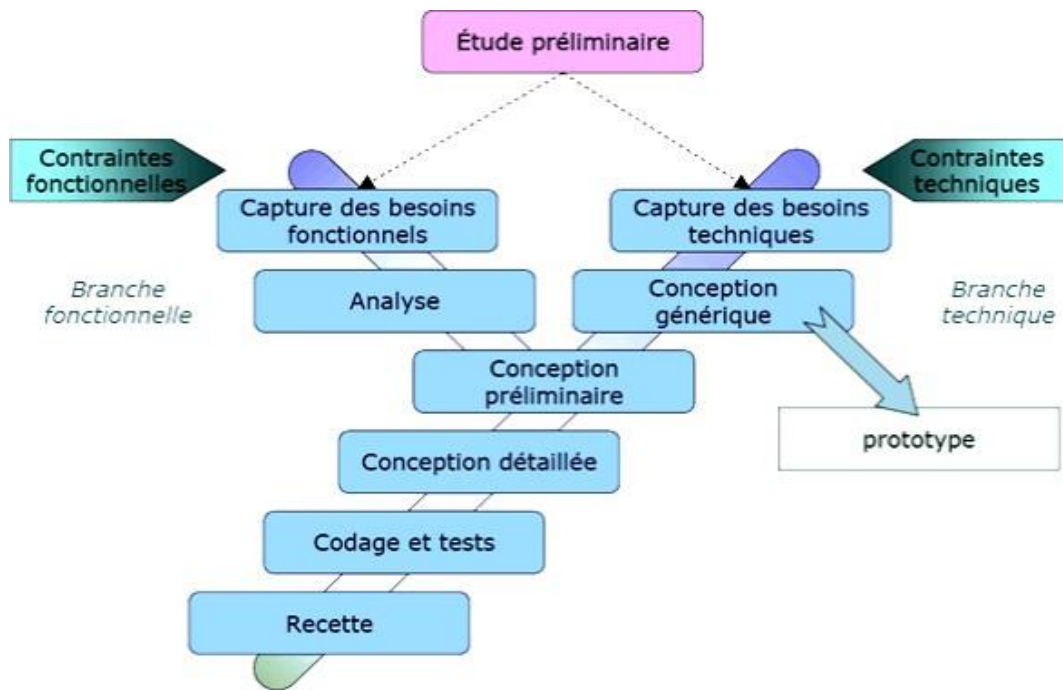


Figure 3 : Schéma illustratif du processus 2TUP⁵

Comme l'illustre la *Figure 3*, le processus 2TUP débute par une étude préliminaire. Cette étape a pour objectif d'identifier les acteurs qui interagiront avec le système, de préciser les échanges entre ces acteurs et le système, d'élaborer un cahier des charges et de modéliser le contexte général du projet.

À la suite de cette étude, le processus se déploie en deux branches distinctes :

➤ **La branche fonctionnelle qui comporte :**

- la capture des besoins fonctionnels, qui produit un modèle des besoins focalisé sur le métier. Cette capture des besoins permet d'éviter au plus tôt le risque de produire un système inadapté aux utilisateurs.
- l'analyse, qui consiste à étudier les spécifications fonctionnelles dans le but d'avoir une idée de ce que va réaliser le système en termes de métier.

Les résultats de l'analyse ne dépendent d'aucune technologie particulière.

➤ **La branche technique qui comporte :**

- la capture des besoins techniques qui consiste à énumérer les outils, matériels et technologies dont nous avons besoin. Elle permet de prendre en compte des

⁵ Source : <https://www.memoireonline.com/05/13/7195/Mise-en-place-dune-application-webmapping-de-geolocalisation-des-points-dintert-de-la-vill7.png>

contraintes (intégration avec l'existant). L'ensemble permet de construire une architecture technique ;

- la conception générique qui consiste à indiquer les composants essentiels à l'élaboration de l'architecture technique. Elle assure la réponse aux exigences du système.

Ces deux branches se fusionnent pour donner la branche du milieu qui comporte :

- **la conception préliminaire** qui fusionne le modèle d'analyse et l'architecture technique afin de structurer le système en composants délivrant des services fonctionnels et techniques ;
- **la conception détaillée** qui décrit de façon précise chacun des composants du système ;
- **le codage, puis tests**, qui consistent à implémenter les fonctionnalités et à les valider progressivement ;
- **la recette**, qui correspond à la validation finale des fonctionnalités développées par rapport aux besoins exprimés dans le cahier des charges.

Avec cette étape méthodologique clarifiée, nous passons à présent au langage de modélisation, qui permet de formaliser l'analyse et la conception du système.

b). Langage de modélisation

Un langage de modélisation offre un cadre formel pour représenter la structure, le comportement et les interactions d'un système. Son utilisation est essentielle pour organiser, documenter et valider les concepts avant toute implémentation. Nous présentons dans le Tableau 2 un aperçu de quelques-uns des langages de modélisation les plus reconnus .

Tableau 2 : Comparaison de quelques langages de modélisation

Langages de modélisation	Caractéristiques
BPMN⁶ (Business Process Model and Notation)	• Modèle de processus pour représenter le déroulement des processus internes ou publics d'une organisation ; • vise l'analyse et la conception des processus métiers qui font intervenir et interagir des systèmes ; • son but principal est de fournir une notation qui soit facilement compréhensible par tous les utilisateurs de l'entreprise.
MBT⁷ (Model Based Testing)	• Déclinaison des solutions d'ingénierie des modèles appliquées au test ; • permet de produire des cas de tests à partir d'un modèle ; • diminue les efforts de maintenance des jeux de tests ; • renforce la qualité de la documentation des exigences.
SysML⁸ (Systems Modeling Language)	• Langage de modélisation spécifique au domaine de l'ingénierie système ; • sémantique plus riche et flexible ; • langage plus réduit que UML.
UML⁹ (Unified Modeling Language)	• Langage de représentation d'un système d'information ; • système de notation orienté objet ; • plus orienté vers la conception ; • international.

Parmi les tous ces langages nous retenons **UML** en raison de son universalité, de la richesse de sa notation et surtout de sa compatibilité naturelle avec le processus 2TUP. Il constitue ainsi l'outil le mieux adapté pour formaliser l'analyse et la conception de notre système. Nous présentons plus en détail le langage UML en annexes (ANNEXE 1 : Langage UML).

⁶ BPMN : <https://www.omg.org/bpmn>

⁷ MBT : https://fr.wikipedia.org/wiki/Model-based_testing

⁸ SysML : <https://www.schoolmouv.fr/cours/le-sysml/fiche-de-cours>

⁹ UML : <https://www.omg.org/spec/UML>

3. Présentation du groupe de travail

Le groupe de travail regroupe l'ensemble des acteurs impliqués dans la mise en œuvre du projet. Il se compose de trois entités principales : le groupe de pilotage, le groupe des utilisateurs et le groupe projet.

a). Groupe de pilotage

Ce groupe assure la direction et la supervision du projet. Il est chargé de planifier les étapes clés, d'évaluer les propositions émises par le groupe projet et de valider les choix stratégiques. Il est constitué de :

- M. Vivian BOGNINI, notre Maître de stage ;
- Dr Yaya TRAORE, notre Directeur de mémoire.

b). Groupe des utilisateurs

Ce groupe rassemble les futurs utilisateurs du système. Leur mission consiste à identifier et préciser les besoins fonctionnels afin d'assurer la conformité de la solution aux réalités de l'entreprise. Il est constitué principalement des responsables métiers et du personnel de ESOF BURKINA.

c). Groupe de projet

Le groupe de projet est chargé de la réalisation effective du projet et sert d'intermédiaire entre le groupe de pilotage et le groupe des utilisateurs. Ce groupe est composé essentiellement de nous-même, Abdoul Razack OUEDRAOGO, étudiant en Troisième année de MIAGE à l'IBAM.

Après avoir identifié les différents intervenants du projet, nous présentons à présent le planning de réalisation.

4. Planning de travail

Pour mener à bien notre travail dans le respect des délais, nous subdivisons le travail en plusieurs tâches spécifiques. À cet effet, nous utilisons le diagramme de Gantt, un outil de planification et d'ordonnancement qui permet de représenter visuellement la répartition des tâches dans le temps et leurs éventuelles dépendances. Le planning prévisionnel qui en résulte est présenté dans la *Figure 4*.

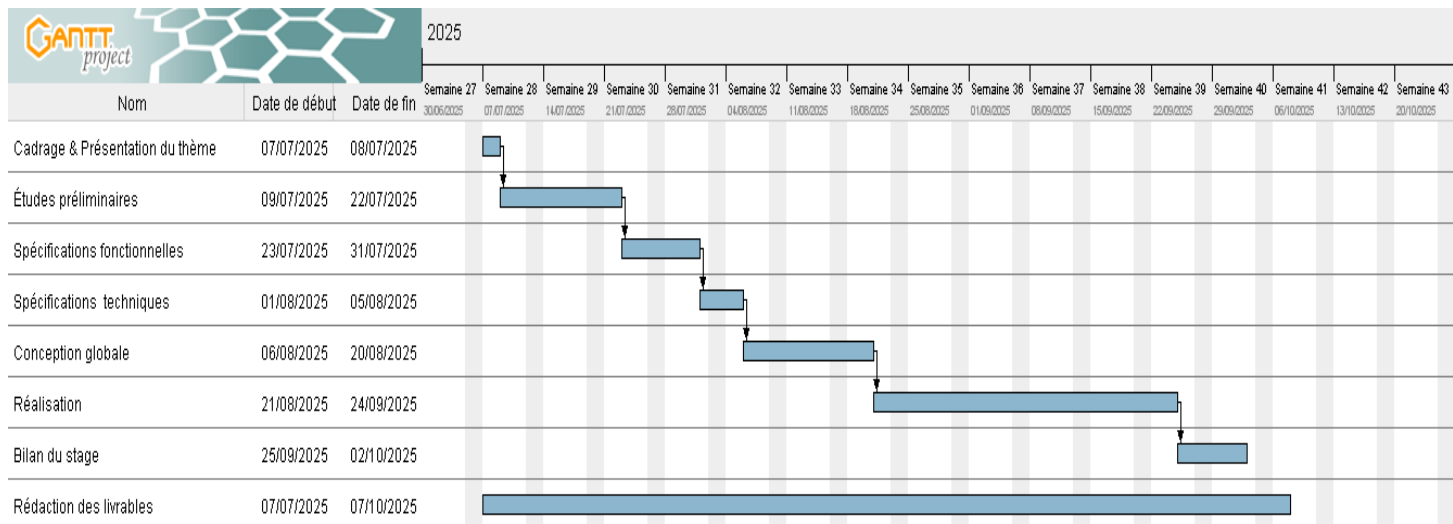


Figure 4 : Planning prévisionnel du projet

À l'issue de la présentation du planning de réalisation, il convient désormais de s'intéresser à l'expression des besoins, en mettant en évidence les acteurs concernés ainsi que les échanges entre ces acteurs et le système.

II. Expression des besoins

L'expression des besoins est une étape essentielle de la conception d'un système d'information. Elle consiste à analyser l'existant et préciser les attentes des utilisateurs afin de proposer une solution adaptée.

1. Étude de l'existant

Pour mener à bien notre travail, nous avons débuté par une étude de l'existant. Cette phase cruciale vise à comprendre et analyser en détail le système actuel de gestion des commandes et des factures chez ESOFT BURKINA. Nous avons ainsi rencontré les responsables de l'entreprise afin de bien comprendre les processus métiers. Cette immersion nous a permis de cartographier le processus et d'identifier les outils informatiques utilisés ainsi que leur limite. Dans cette partie, nous dressons un état des lieux en décrivant les pratiques en vigueur, le système utilisé et les limites observées.

a). Présentation du processus actuel

Le processus de commande et facturation à ESOFT BURKINA suit plusieurs étapes successives. Afin d'illustrer le déroulement de ces étapes et faciliter leur compréhension, nous présentons ici deux diagrammes d'activité métier, retraçant, d'une part, le cycle allant de la

commande jusqu'à la facture client dans la *Figure 5* et, d'autre part, le processus lié aux commandes passées auprès des fournisseurs dans la *Figure 6*.

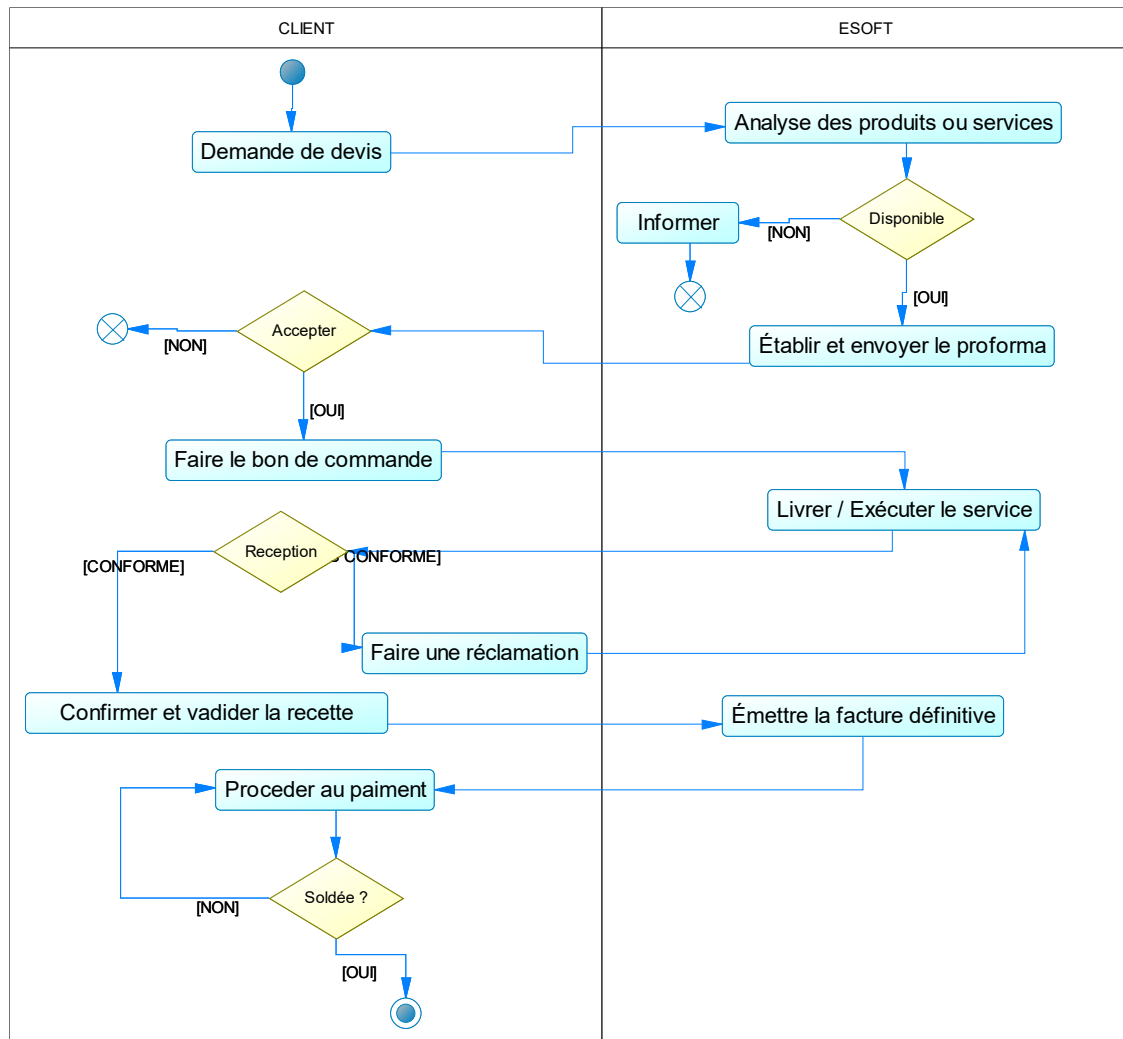


Figure 5 : Processus de commande et de facturation (Vue Client / ESOFT)

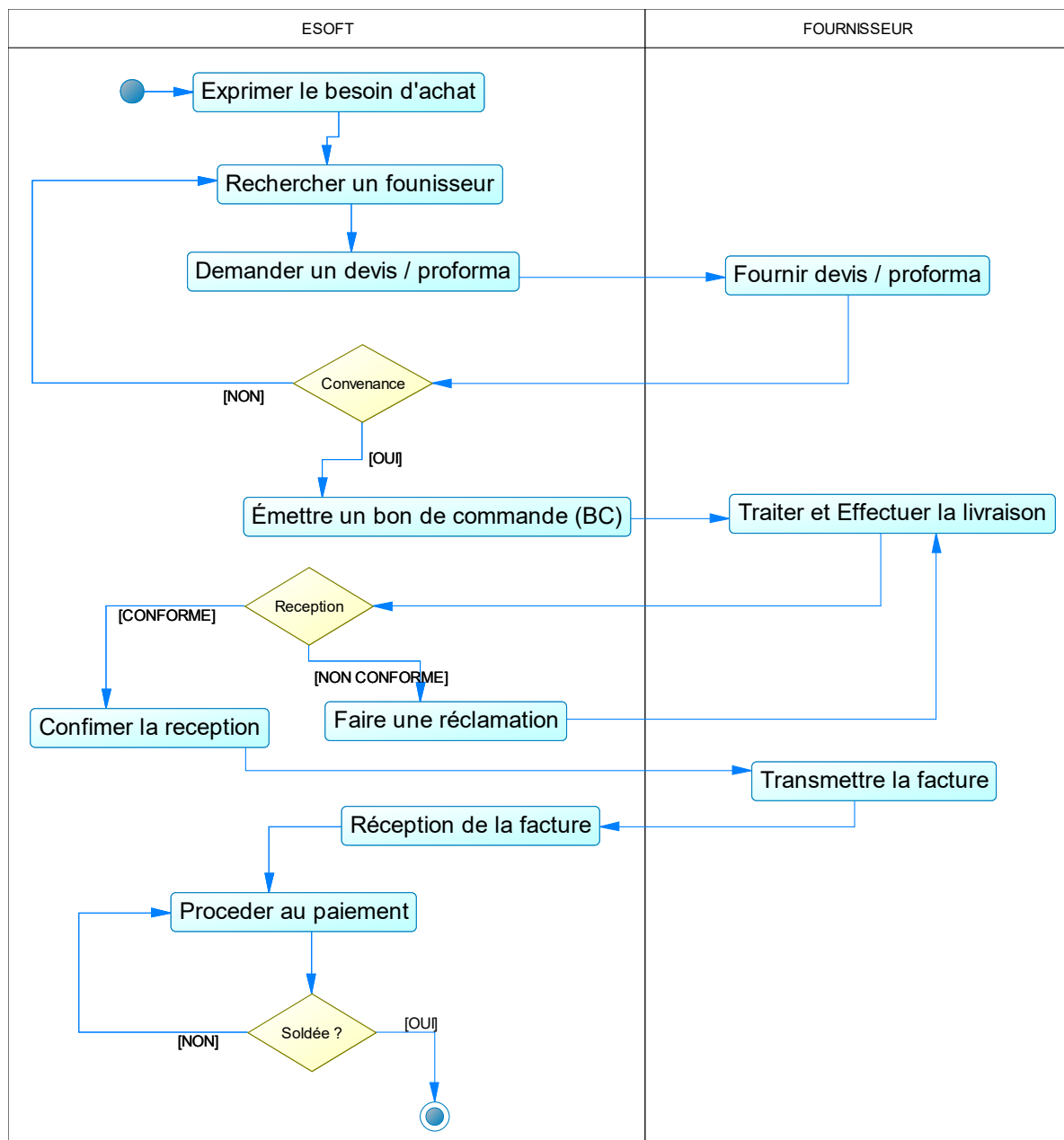


Figure 6 : Processus de commande et de facturation (Vue ESOFT / Fournisseur)

Ces diagrammes permettent de mieux comprendre le fonctionnement actuel et de mettre en évidence les points clés du processus. Nous passons maintenant à une présentation des outils informatiques utilisés actuellement par l'entreprise pour piloter ces différents processus.

b). Présentation du dispositif actuel

Pour assurer la gestion de ses différents processus métiers, l'entreprise s'appuie sur une instance professionnelle **Odoo**¹⁰, un Progiciel de Gestion Intégré (ERP)¹¹ open source et accessible en ligne. Cet outil est fondamental dans l'organisation actuelle, car il assure la centralisation des données et la couverture de fonctions clés telles que la gestion de la relation client (CRM)¹², la gestion des ventes, des achats et de la facturation de base. L'instance d'Odoo utilisée actuellement permet notamment la génération automatisée des documents essentiels tels que les proformas, les bons de commande, les bordereaux de livraison et les factures définitives pour les clients.

c). Limites du dispositif actuel

Malgré l'utilisation d'un ERP généraliste tel qu'Odoo, certains besoins spécifiques mettent en évidence plusieurs limites :

- le suivi détaillé des règlements de factures demeure complexe, en particulier lorsqu'il s'agit de paiements partiels ou multiples pour une même facture ;
- les retenues ne sont pas intégrées au processus de paiement de manière fluide ;
- les garanties de bonne exécution associées aux factures ne sont pas prises en charge ;
- il n'existe pas de tableau de bord consolidé permettant une vision globale et en temps réel des créances et des dettes.

En raison de ces limites, l'entreprise est souvent contrainte de recourir à des fichiers Excel et à des traitements manuels pour y remédier, ce qui fragmente les données et nuit à la fiabilité des informations.

2. Exigences fonctionnelles

Suite à l'analyse de l'existant et à des entretiens avec les responsables de l'entreprise, nous avons recensé les exigences fonctionnelles auxquelles la solution doit répondre.

Les principales exigences sont les suivantes :

➤ Gestion des clients et fournisseurs

- enregistrer, mettre à jour et consulter les informations des clients et des fournisseurs.

¹⁰ Odoo : https://www.odoo.com/fr_FR

¹¹ ERP : Enterprise Resource Planning

¹² CRM : Customer Relationship Management

➤ **Gestion des factures**

- enregistrer et suivre les factures clients et fournisseurs ;
- gérer automatiquement l'évolution du statut des factures ;
- assurer la traçabilité complète de chaque facture depuis son enregistrement jusqu'à son règlement.

➤ **Gestion des règlements de facture client et fournisseur**

- enregistrer les règlements effectués par virement, chèque ou autre ;
- permettre la traçabilité de chaque paiement;
- suivre les paiements partiels et calculer automatiquement le solde restant.

➤ **Gestion des retenues**

- enregistrer les retenues à la source appliquées par certains clients ;
- calculer automatiquement le montant net perçu et le montant restant dû ;
- justifier chaque retenue par un motif.

➤ **Gestion des garanties de bonne exécution**

- associer une garantie à une facture ;
- suivre le montant garanti, la date de début et la date d'échéance ;
- générer des alertes à l'approche de l'expiration de la garantie.

➤ **Tableau de bord consolidé**

- présenter en temps réel la situation des créances et dettes ;
- visualiser les factures en attente, partiellement réglées ou échues ;
- fournir une vision globale et synthétique du suivi des factures, paiements et garanties.

➤ **Reporting et exportation**

- générer des états récapitulatifs sur les factures, paiements, retenues et garanties ;
- exporter les rapports en formats standards (PDF, Excel) ;
- offrir une historisation et une traçabilité complète des opérations.

Nous retenons ces exigences comme base de référence pour orienter la solution que nous proposons.

3. Présentation du processus de fonctionnement cible

À partir des exigences fonctionnelles définies précédemment, nous avons étudié plusieurs pistes afin de proposer une approche efficace. L'objectif est de proposer une approche qui réponde efficacement aux besoins exprimés, tout en tenant compte de ses contraintes organisationnelles et techniques de l'entreprise.

a). Solutions envisageables

Pour combler les insuffisances relevées et répondre aux exigences fonctionnelles, trois options principales se présentent :

- **(1) améliorer Odoo via des modules personnalisés** : cette option consiste à développer des modules spécifiques dans Odoo en utilisant son framework afin de couvrir les besoins non satisfaits ;
- **(2) recourir à d'autres outils du marché** : il s'agit d'adopter une solution logicielle alternative offrant nativement les fonctionnalités requises
- **(3) concevoir un système complémentaire sur mesure** : Cette approche consiste à mettre en place une application spécifiquement adaptée au mode de fonctionnement de l'entreprise.

Avant de présenter la solution retenue, nous présentons, à travers le Tableau 3, quelques logiciels disponibles pouvant potentiellement répondre à certains nos exigences.

Tableau 3 : Évaluation comparative d'autres logiciels de facturation

Exigences \ Outils	Zoho Books ¹³	Xero ¹⁴	FreshBooks ¹⁵	Sage Intacct ¹⁶	NetSuite ¹⁷
Factures clients & fournisseurs (statuts)	Oui (surtout clients)	Oui (surtout clients)	Oui (clients)	Oui	Oui
Gestion des règlements (virement/chèque) & traçabilité	Oui (via intégrations)	Oui	Oui	Oui	Oui

¹³ Zoho Books ; <https://www.zoho.com/books/>

¹⁴ Xero : <https://www.xero.com/>

¹⁵ FreshBooks : <https://www.freshbooks.com/>

¹⁶ Sage Intacct : <https://www.sage.com/intacct>

¹⁷ NetSuite : <https://www.netsuite.com/>

Enregistrement paiements partiels (ventilation par facture)	Non	Partiel	Partiel	Oui	Oui
Retenues complexes	Non	Non	Non	Non	Partiel
Garanties de bonne exécution (GBE)	Non	Non	Non	Non	Partiel
Dashboard consolidé créances/dettes	Partiel	Partiel	Partiel	Oui	Oui
Reporting & export (PDF/Excel)	Oui	Oui	Oui	Oui	Oui
Traçabilité / historisation des opérations	Partiel	Partiel	Partiel	Oui	Oui
Interopérabilité / API avec outils existants	Oui	Oui	Oui	Oui	Oui
Simplicité d'usage (PME)	Oui	Oui	Oui	Non	Non
Coût estimatif (par mois/utilisateur)	≈ 6 000-12 000 FCFA	≈ 12 000-18 000 FCFA	≈ 9 000-15 000 FCFA	Sur devis (souvent > 80 000 FCFA)	Sur devis (souvent > 150 000 FCFA)

À l'issue de cette étude, nous constatons que les autres solutions logicielles du marché ne couvrent pas entièrement les besoins spécifiques de ESOFT BURKINA. L'entreprise a aussi exprimé sa volonté de disposer d'une solution personnelle et évolutive, capable de s'adapter à de futurs besoins métiers.

Nous retenons donc **la conception d'un système complémentaire sur mesure**, interfaçable avec Odoo. Ce choix offre :

- un suivi centralisé des factures, règlements, retenues et garanties ;
- une intégration à l'ERP existant tout en corrigeant ses insuffisances ;
- une flexibilité permettant d'évoluer avec les futurs besoins métiers.

Cette orientation permet à ESOFT BURKINA de renforcer la fiabilité et la traçabilité de ses processus financiers, tout en garantissant une meilleure adéquation entre les outils numériques et ses pratiques de gestion réelles.

Dans la suite de notre travail, nous présentons les spécifications fonctionnelles du futur système, afin de décrire de manière détaillée son fonctionnement.

4. Spécifications fonctionnelles

Les spécifications fonctionnelles décrivent les fonctionnalités attendues du futur système, en précisant les acteurs, les cas d'utilisation et les interactions principales.

a). Identification des acteurs

Un acteur définit un ensemble cohérent de rôles qu'un utilisateur ou une entité externe peut jouer en interagissant avec le système. Dans notre cas, deux acteurs principaux sont identifiés :

- **l'administrateur** : responsable de la gestion globale de la plateforme. Il dispose de tous les droits, notamment la création et la gestion des comptes utilisateurs, la configuration du système et la supervision des activités;
- **l'utilisateur** : acteur opérationnel chargé des tâches quotidiennes telles que l'enregistrement des factures, des paiements et des garanties, ainsi que le suivi de leur évolution dans le temps ;

b). Cas d'utilisation

Les cas d'utilisation désignent l'ensemble des interactions qui permettent à un acteur d'atteindre son objectif en utilisant le système. Le *Tableau 4* ci-dessous présente l'ensemble des cas d'utilisation de notre système.

Tableau 4: Listes des cas d'utilisation

Numéro	Cas d'utilisation	Description	Acteurs
CU_01	S'authentifier	Permet à un acteur de se connecter au système pour accéder à ses fonctionnalités.	Tous les acteurs
CU_02	Gérer les utilisateurs	Permet de créer, modifier ou supprimer les comptes afin d'assurer le contrôle des accès à la plateforme.	Administrateur

<i>CU_03</i>	Configurer les paramètres système	Permet de paramétrer les options globales et d'assurer la supervision générale de la plateforme.	Administrateur
<i>CU_04</i>	Gérer fournisseurs	Permet d'ajouter, de mettre à jour, ou de supprimer les informations des fournisseurs.	Utilisateur
<i>CU_05</i>	Gérer Clients	Permet d'ajouter, de mettre à jour, ou de supprimer les informations des clients.	Utilisateur
<i>CU_06</i>	Gérer garanties de bonne exécution	Permet d'ajouter, de mettre à jour, supprimer les garanties de bonne exécution.	Utilisateur
<i>CU_07</i>	Enregistrer Facture Fournisseur.	Permet d'enregistrer une facture reçue d'un fournisseur afin de suivre les règlements à effectuer.	Utilisateur
<i>CU_08</i>	Enregistrer Facture Client.	Permet de d'enregistrer une facture remise à un client pour en assurer le suivi.	Utilisateur
<i>CU_09</i>	Enregistrer un règlement fournisseur.	Permet de renseigner le règlement fait à un fournisseur avec les détails.	Utilisateur
<i>CU_10</i>	Enregistrer un règlement client.	Permet de renseigner un règlement fait par un client avec les détails.	Utilisateur
<i>CU_11</i>	Consulter factures.	Permet de consulter l'état, l'évolution et les paiements des factures.	Utilisateur
<i>CU_12</i>	Générer des rapports	Permet de générer et d'extraire des rapports récapitulatifs sur les états de dette et de créances, factures, paiements, garanties.	Administrateur

c). Diagramme des cas d'utilisation

Après avoir identifié et défini les différents acteurs et les cas d'utilisation du système, il est nécessaire de les représenter sous une forme schématique, afin d'en faciliter la compréhension. Le diagramme des cas d'utilisation UML constitue ainsi l'outil adapté pour cette représentation. Il permet de visualiser, sous une forme graphique, l'ensemble des interactions possibles entre les acteurs et le système. Cette représentation offre une vue d'ensemble claire et synthétique du fonctionnement futur du système. La *Figure 7* illustre le diagramme de cas d'utilisation correspondant.

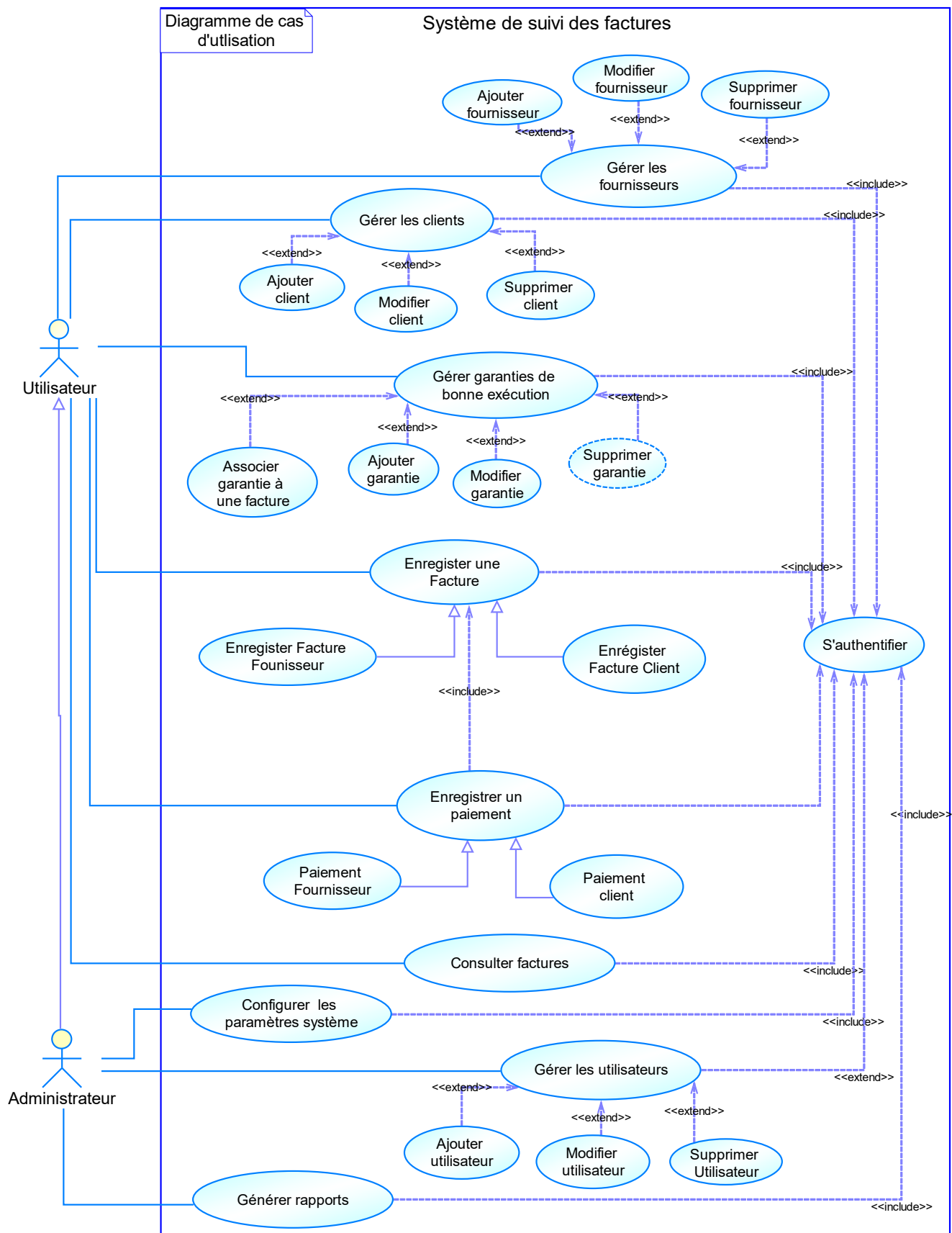


Figure 7: Diagramme des cas d'utilisation

d). Description textuelle de certains cas d'utilisation

Après avoir défini les grandes fonctions du système du point de vue des acteurs à travers le diagramme des cas d'utilisation, la description textuelle a pour rôle d'exposer et de documenter de manière détaillée le dialogue entre les acteurs et le système. À cet effet, chaque cas d'utilisation est présenté sous forme de tableau comprenant plusieurs rubriques :

- **acteurs** : liste des utilisateurs ou entités externes impliqués dans le cas d'utilisation ;
- **précondition** : état requis du système avant le déclenchement du cas d'utilisation ;
- **scénario nominal** : déroulement normal des interactions entre l'acteur et le système sans erreur ;
- **scénario alternatif** : variantes du scénario nominal en cas d'erreurs ou d'événements particuliers ;
- **postcondition** : état final du système après l'exécution du cas d'utilisation.

Compte tenu de la diversité de nos cas d'utilisation, seules quelques descriptions de cas essentiels sont présentées ici afin de faciliter la lecture de notre rapport. Il s'agit notamment de : « **S'authentifier** », « **Enregistrer Facture Client** », « **Associer une garantie de bonne exécution** », « **Enregistrer un paiement client** » respectivement dans *Tableau 5*, *Tableau 6*, *Tableau 7* et *Tableau 8*.

□ **CU 01 : S'authentifier**

Tableau 5: Description textuelle du cas d'utilisation « S'authentifier »

Résumé	Permet à un acteur de se connecter au système pour accéder à ses fonctionnalités.
Acteurs	Tous les acteurs.
Précondition	- Disposer d'un compte enregistré et actif. - Disposer d'un navigateur et une connexion internet.
Scénario nominal	1. L'acteur accède à la page de connexion. 2. Le système affiche le formulaire d'authentification. 3. L'acteur saisit ses identifiants et valide. 4. Le système vérifie les informations d'identification [A]. 5. Le système authentifie l'utilisateur. 6. Le système ouvre une session et redirige l'utilisateur vers son tableau de bord.
Scénario alternatif	[A] : Nom d'utilisateur ou mot de passe incorrect a) Le système affiche un message d'erreur. b) Le formulaire est reproposé.

	c) Le scénario reprend à l'étape 3
Postcondition	L'utilisateur est authentifié et redirigé vers son tableau de bord

Ce tableau décrit le déroulement complet du processus d'authentification, depuis la saisie des identifiants jusqu'à la redirection vers le tableau de bord. Il permet d'explicitier clairement les conditions, les étapes normales et les cas d'erreurs associés à cette action essentielle du système.

□ **CU02 : Enregistrer une facture client**

Tableau 6 : : Description textuelle du cas d'utilisation « Enregistrer une facture client »

Résumé	Permet d'enregistrer une facture client dans le système
Acteurs	Utilisateur
Précondition	L'utilisateur est connecté
Scénario nominal	1. L'utilisateur choisit l'option « Enregistrer facture client ». 2. Le système affiche le formulaire d'enregistrement. 3. L'utilisateur saisit la référence de la facture. 4. Le système vérifie si la facture existe déjà [A1]. 5. Le système interroge Odoo pour rechercher la facture [A2]. 6. Les informations de la facture sont préremplies dans le formulaire. 7. L'utilisateur complète les champs et soumet le formulaire. 8. Le système enregistre la facture et confirme l'opération
Scénario alternatif	[A1] : Facture déjà enregistrée a) Le système affiche un message d'avertissement et annule l'opération. [A2] : Facture non trouvée sur Odoo a) L'utilisateur saisit manuellement toutes les informations. b) Le scénario reprend à l'étape 7.
Postcondition	La facture client est enregistrée dans le système et disponible pour suivi.

Ce tableau détaille les étapes nécessaires pour enregistrer une facture client, que les informations proviennent d'Odoo ou soient saisies directement. Il montre comment le système gère les doublons, les erreurs et le déroulement normal de l'opération.

□ **CU07 : Associer une garantie de bonne exécution**

Tableau 7 : Description textuelle du cas d'utilisation « Associer une garantie de bonne exécution »

Résumé	Permet de lier une garantie de bonne exécution à une facture client.
Acteurs	Utilisateur
Précondition	- L'utilisateur est connecté - La facture est enregistrée dans la base de données.
Scénario nominal	1. L'utilisateur choisit l'option « Associer garantie » sur une facture. 2. Le système charge les garanties disponibles. 3. L'utilisateur sélectionne une garantie et valide [A1]. 4. Le système rattache la garantie à la facture. 5. Une confirmation est affichée.
Scénario alternatif	[A1] : Garantie absente de la liste a) L'utilisateur crée directement une nouvelle garantie b) Le système enregistre la nouvelle garantie. c) L'utilisateur la sélectionne et valide. d) Le scénario reprend à l'étape 4.
Postcondition	La garantie est enregistrée et associée à la facture.

Ce tableau décrit les étapes nécessaires pour rattacher une garantie à une facture et précise comment le système réagit lorsqu'aucune garantie existante ne peut être sélectionnée. Il met en évidence le déroulement nominal ainsi que la création éventuelle d'une nouvelle garantie.

□ **CU08 : Enregistrer un paiement client**

Tableau 8 : Description textuelle du cas d'utilisation « Enregistrer un paiement client ».

Résumé	Permet d'enregistrer le règlement d'une facture par le client et de mettre à jour son statut.
Acteurs	Utilisateur
Précondition	- Une facture client existe et n'est pas totalement réglée. - L'utilisateur est connecté.

Scénario nominal	1. L'utilisateur accède au module « Paiements client » 2. Le système liste les factures non réglées. 3. L'utilisateur sélectionne la facture concernée. 4. Le système affiche les détails. 5. L'utilisateur saisit les informations du paiement. 6. L'utilisateur renseigne les retenues éventuelles [A1]. 7. L'utilisateur valide. 8. Le système enregistre le paiement et met à jour la facture [A2]. 9. Une confirmation est affichée.
Scénario alternatif	[A1] : Retenue appliquée a) Le système calcule le montant net perçu et le reste dû. [A2] : Erreur de saisie a) Le système affiche un message d'erreur. b) L'utilisateur corrige et reprend à l'étape 7.
Postcondition	Le paiement est enregistré et le statut de la facture est mis à jour

Ce tableau expose le déroulement de l'enregistrement d'un paiement client et la mise à jour de la facture correspondante. Il décrit aussi la gestion des retenues et les corrections possibles en cas d'erreur de saisie.

e). Diagramme de séquences

Le diagramme de séquence UML est un outil qui complète la description textuelle en mettant l'accent sur la dimension temporelle des cas d'utilisation. Il représente graphiquement l'enchaînement chronologique des interactions à travers les messages échangés entre les acteurs et les composants du système. Cette représentation permet de visualiser pas à pas le déroulement d'un scénario.

Ici, nous présentons les diagrammes de séquence des cas d'utilisation décrits textuellement précédemment.

CU 01 : S'authentifier

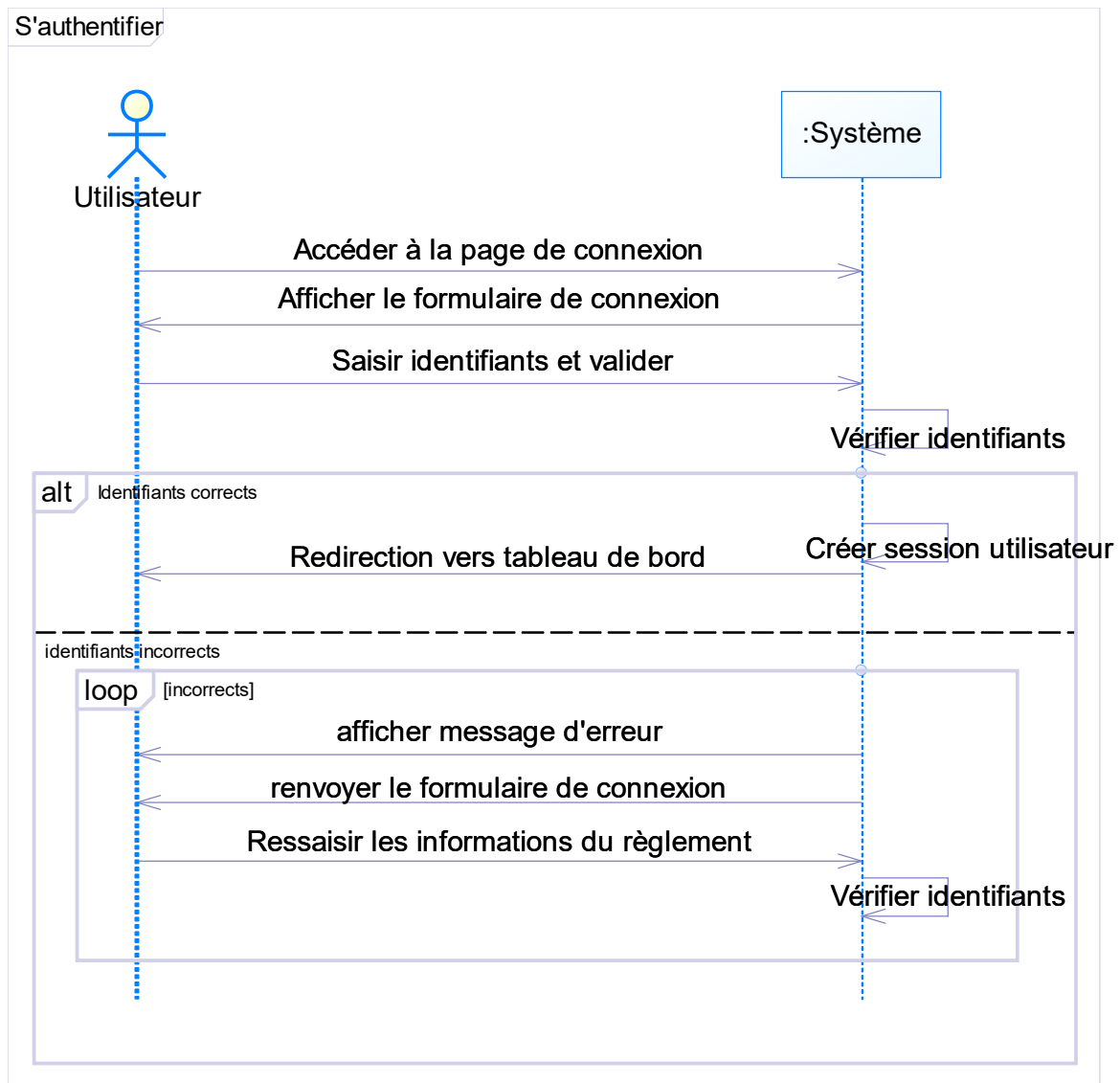


Figure 8 : Diagramme de cas de séquence « S'authentifier »

Ce diagramme illustre le déroulement complet du processus d'authentification entre l'utilisateur et le système, en montrant l'enchaînement des messages échangés. Il met également en évidence les deux situations possibles : la validation correcte des identifiants et la reprise du processus en cas d'erreur de saisie.

□ **CU 02 : Enregistrer une facture client**

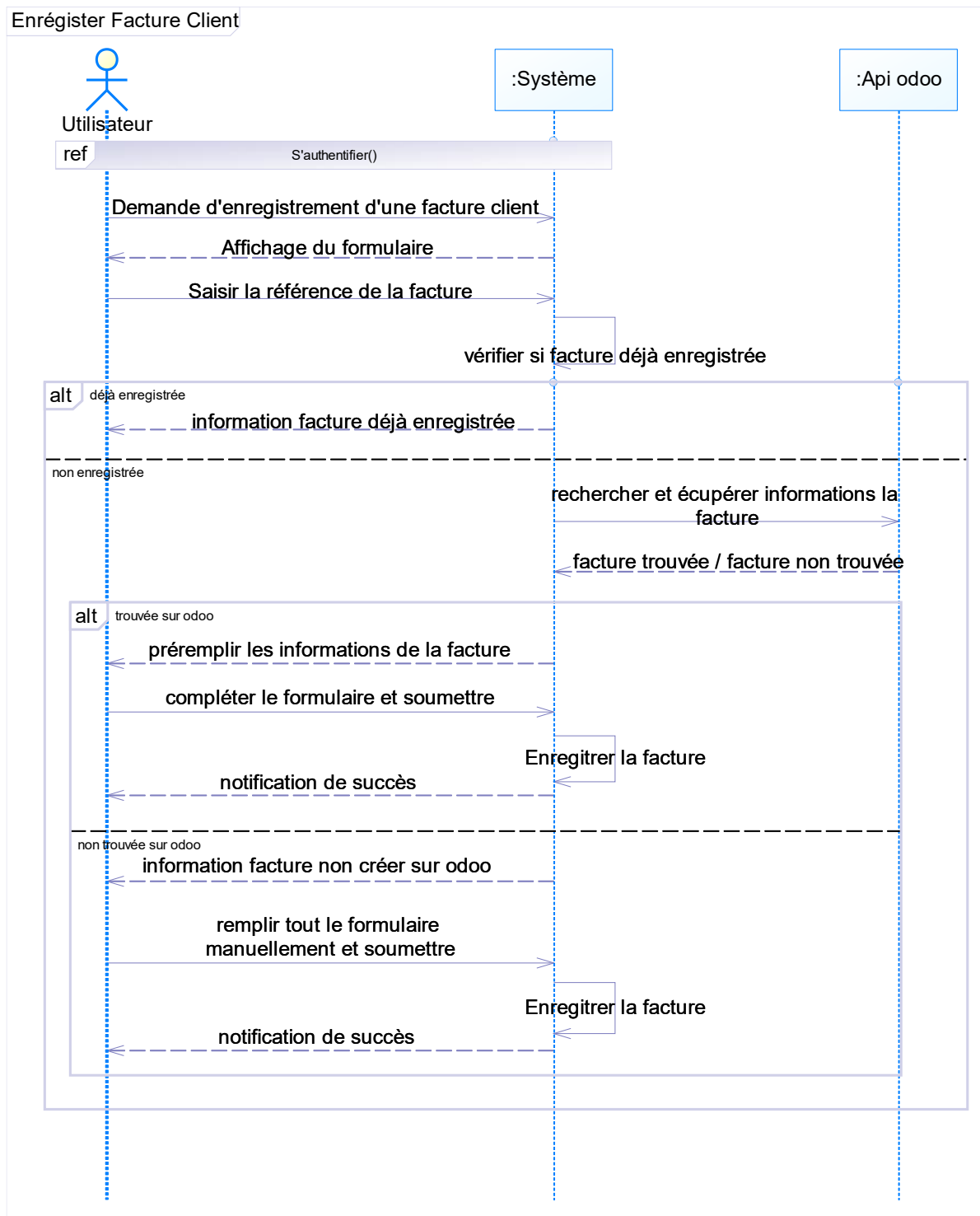


Figure 9 : Diagramme de séquence « Enregistrer une facture client »

Ce diagramme montre l'enchaînement des interactions entre l'utilisateur, le système et l'API Odoo lors de l'enregistrement d'une facture. Il met en évidence les deux scénarios possibles : récupération automatique des informations via Odoo ou saisie directe manuelle lorsque la facture n'est pas trouvée via l'API Odoo.

□ **CU 07 : Associer une garantie de bonne exécution**

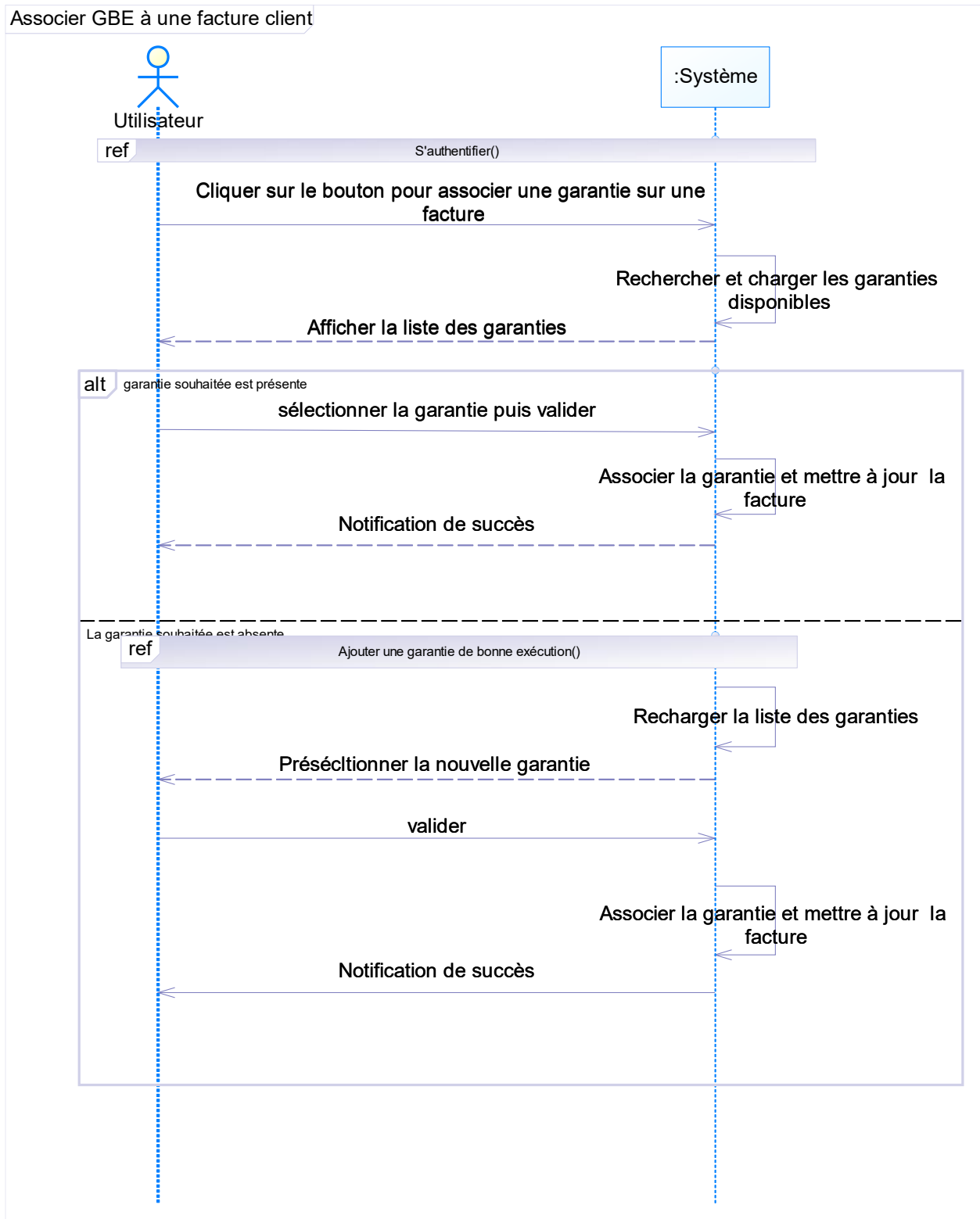


Figure 10 : Diagramme de cas de séquence « Associer une garantie de bonne exécution »

Ce diagramme présente les interactions entre l'utilisateur et le système lorsqu'une garantie de bonne exécution est associée à une facture. Il montre à la fois le déroulement normal et le cas où aucune garantie existante ne correspond, nécessitant alors une création et une sélection dans la même séquence.

□ **CU 08 : Enregistrer un paiement client**

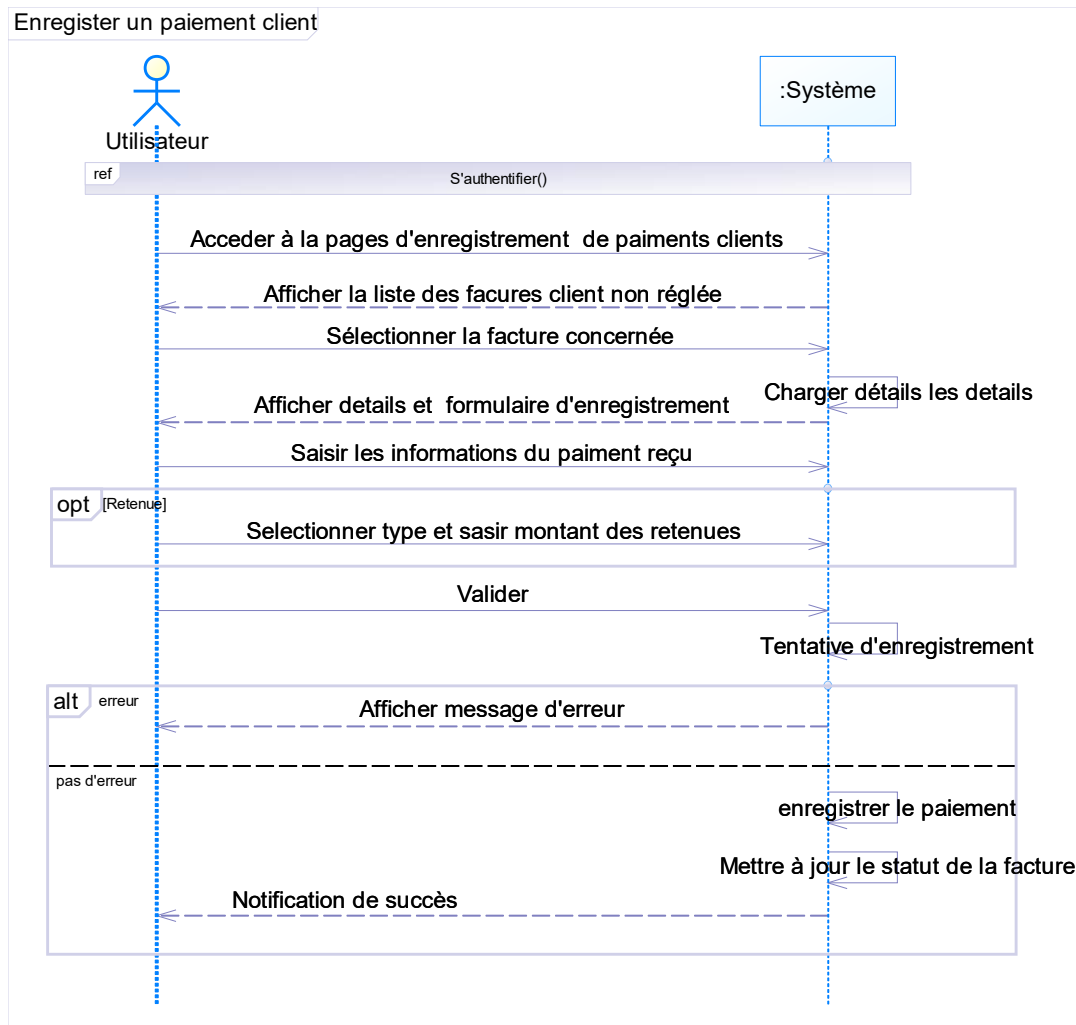


Figure 11 : Diagramme de séquence « Enregistrer un paiement client »

5. Spécifications techniques

Conformément à la démarche 2TUP, la phase de spécifications techniques vise à traduire les choix issus de l'analyse fonctionnelle en solutions techniques concrètes et opérationnelles. Elle consiste à définir l'architecture cible du système, l'environnement matériel et logiciel, ainsi que les outils nécessaires à sa mise en œuvre.

a). Mise à disposition des conditions de travail

Pour la réalisation de ce projet, nous bénéficions de locaux confortables, d'un bureau et d'une

connexion Wi-Fi mis à notre disposition par ESOFT en journée. Ce cadre de travail propice favorise notre organisation et l'efficacité dans la réalisation de nos missions.

b). Architecture de développement

L'architecture de développement définit la manière dont les différents composants d'un système s'organisent et interagissent entre eux. Elle influence directement la stabilité, la performance et l'évolutivité du logiciel. Le choix d'une architecture appropriée permet de garantir que la solution développée réponde efficacement aux besoins exprimés. Il existe plusieurs styles d'architecture, notamment les modèles **monolithique**, **2-tiers**, **3-tiers** et **microservices**. Le

Tableau 9 présente une étude comparative de ces différentes approches afin de mettre en évidence leurs avantages et leurs limites et à justifier l'architecture retenue pour notre système.

Tableau 9 : Comparaison des différentes architectures de développement

Architecture	Description	Avantages	Limites
Monolithique ¹⁸	Application unique regroupant toutes les fonctionnalités.	Déploiement simple, cohérence forte.	Difficultés d'évolution, risques élevés en cas de panne.
2-tiers (Client-Serveur) ¹⁹	L'application est divisée en deux parties : une interface utilisateur et une base de données centrale.	Simplicité, rapidité de mise en œuvre.	Peu évolutive, dépendance forte entre client et serveur.
3-tiers ²⁰	Séparation en trois couches : Présentation, Logique métier et Données.	Modularité, évolutivité, sécurité renforcée.	Complexité de mise en place plus élevée.
Microservices ²¹	Ensemble de services indépendants communiquant via API.	Scalabilité, flexibilité technologique, indépendance des modules.	Gestion complexe, coûts élevés, nécessite une équipe expérimentée.

¹⁸ <https://www.ibm.com/think/topics/monolithic-architecture>

¹⁹ <https://www.ituonline.com/tech-definitions/what-is-two-tier-architecture>

²⁰ <https://www.ibm.com/think/topics/three-tier-architecture>

²¹ <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/microservices>

Au regard des besoins identifiés et des contraintes techniques, nous avons retenu **une architecture 3-tiers**. Elle offre une séparation claire des responsabilités, favorise la modularité et garantit une intégration fluide avec l'ERP Odoo déjà utilisé. La *Figure 12* illustre le fonctionnement d'une architecture 3-tiers.

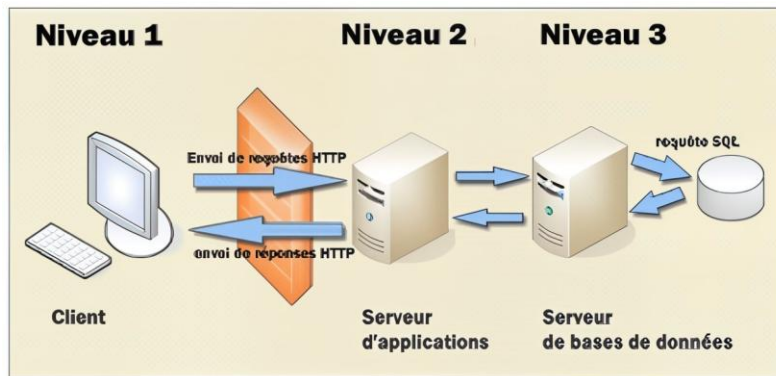


Figure 12 : Architecture 3-tiers : Schéma illustratif²²

Une fois l'architecture définie, nous passons à la conception globale de notre système.

III. Conception globale

Dans cette partie de notre document nous aborderons la conception à proprement parler, nous présentons le diagramme de classe, de déploiement, d'activités et certains diagrammes d'état-transition.

1. Diagramme des classes

Nous présentons d'abord le dictionnaire des données puis le diagramme des classes.

a). Dictionnaire de données

Le dictionnaire de données recense et définit précisément tous les attributs des classes d'un système, ainsi que leurs types de données. Pour faciliter la consultation, le nôtre est organisé par classe, et seules les classes principales sont décrites dans ce chapitre, les autres classes étant détaillées en annexe (ANNEXE 2 : Le dictionnaires des données.).

□ Classe : Structure

²² Source : <https://images.app.goo.gl/V7EJ1jG5Lze3wv2w8>

Tableau 10 : Dictionnaire de données de la classe « Structure »

Attribut	Description	Type de donnée
id	Identifiant de la structure	Entier
email	Adresse e-mail	Chaîne de caractères
nom	Dénomination / raison sociale	Chaîne de caractères
sigle	Sigle de la structure	Chaîne de caractères
adresse	Adresse postale/physique	Chaîne de caractères
estClient	Indique si la structure est un client	Booléen

□ **Classe : Facture**

Tableau 11 : Dictionnaire de données de la classe « Facture »

Attribut	Description	Type de donnée
id	Identifiant de la facture	Entier
reference	Référence ou numéro de la facture	Chaîne de caractères
intitule	Intitulé ou l'objet de la facture	Chaîne de caractères
numBonCommande	Référence du bon de commande	Chaîne de caractères
statut	État de la facture	Chaîne de caractères
tauxTVA	Taux de TVA appliqué	Numérique
montantHT	Montant hors taxes	Numérique
dateEcheance	Date limite de paiement	Date
dateEtablissement	Date d'établissement	Date
dateDepot	Date de dépôt chez le client ou la date de réception s'il s'agit d'une facture fournisseur.	Date
piceJointe	Chemin/réf. de la pièce jointe	Chaîne de caractères

□ **Classe : ReglementFacture**

Tableau 12 : Dictionnaire de données de la classe « ReglementFacture »

Attribut	Description	Type de donnée
id	Identifiant du règlement	Entier
datePaiement	Date effective du paiement	Date

dateEnregistrement	Date d'enregistrement système	Date
montant	Montant réglé	Numérique
pieceJointe	Url du Fichier Justificatif (reçu, virement...)	Chaîne de caractères

□ **Classe : Retenue**

Tableau 13 : Dictionnaire de données de la classe « Retenue »

Attribut	Description	Type de donnée
id	Identifiant de la retenue	Entier
motif	Motif de la retenue	Chaîne de caractères
taux	Taux appliqué (%)	Numérique
retenueTVA	Retenue liée à la TVA	Booléen
pieceJointe	Url du fichier justificatif de retenue	Chaîne de caractères

□ **Classe : GarantieBonneExecution**

Tableau 14 : Dictionnaire de données de la classe « GarantieBonneExecution »

Attribut	Description	Type de donnée
id	Identifiant de la garantie	Entier
libele	Libellé de la garantie	Chaîne de caractères
pourcentage	Pourcentage garanti (ex. 5%)	Numérique
dateDebut	Début de validité	Date
dateFin	Fin/échéance	Date
etatGarantie	État de la garantie	Chaîne de caractères
pieceJointe	Copie/lettre de garantie	Chaîne de caractères

b). Diagramme des classes

Le diagramme de classe est une représentation visuelle de la structure statique d'un système, mettant en évidence les classes, leurs attributs et les relations entre elles. La *Figure 13* ci-dessous illustre notre diagramme de classe.

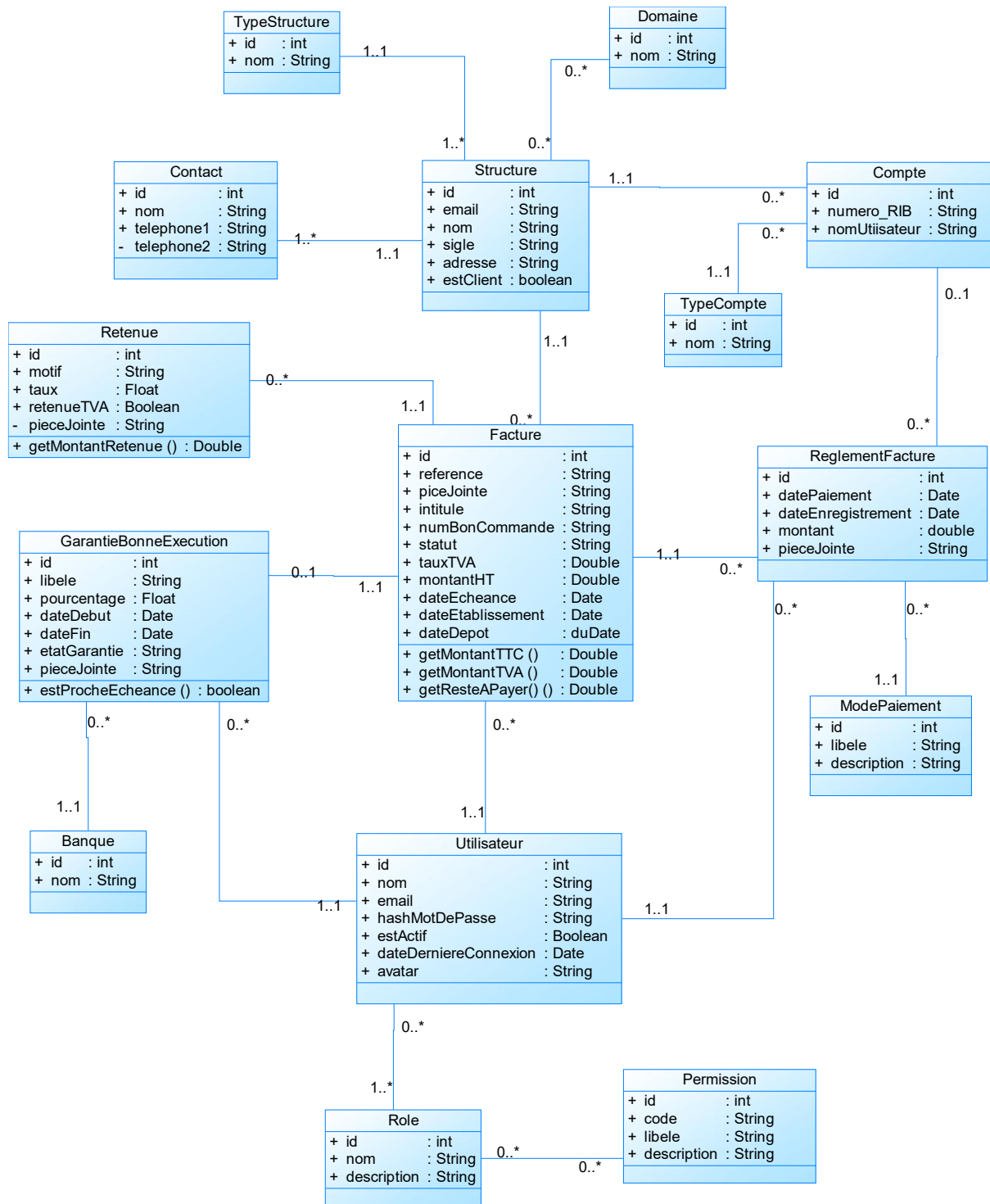


Figure 13 : Le diagramme des classes.

Ce diagramme met en évidence l'organisation logique de l'application autour des entités métier essentielles : factures, paiements, retenues, garanties, structures et utilisateurs. Il montre comment ces éléments interagissent entre eux pour couvrir l'ensemble du cycle de facturation et de suivi dans le système.

Par exemple, la classe « Facture », au cœur du modèle, représente chaque facture enregistrée et ses informations financières. Elle est liée à plusieurs « RèglementFacture », reflétant le fait qu'une facture peut être réglée en plusieurs étapes ; elle peut également être associée à une « GarantieBonneExecution » lorsqu'une garantie est requise ; et elle peut comporter plusieurs « Retenue » appliquées lors des paiements.

2. Diagrammes d'état-transitions

Le diagramme d'état-transitions est un diagramme UML comportemental qui décrit le cycle de vie d'un objet en mettant en évidence les états successifs qu'il peut prendre ainsi que les événements qui déclenchent les transitions. Il est particulièrement adapté pour modéliser des entités dont l'évolution dépend d'événements précis, dans notre cas, il s'agit essentiellement une **facture** ou une **garantie de bonne exécution**.

□ Diagramme d'état-transition d'une Facture client

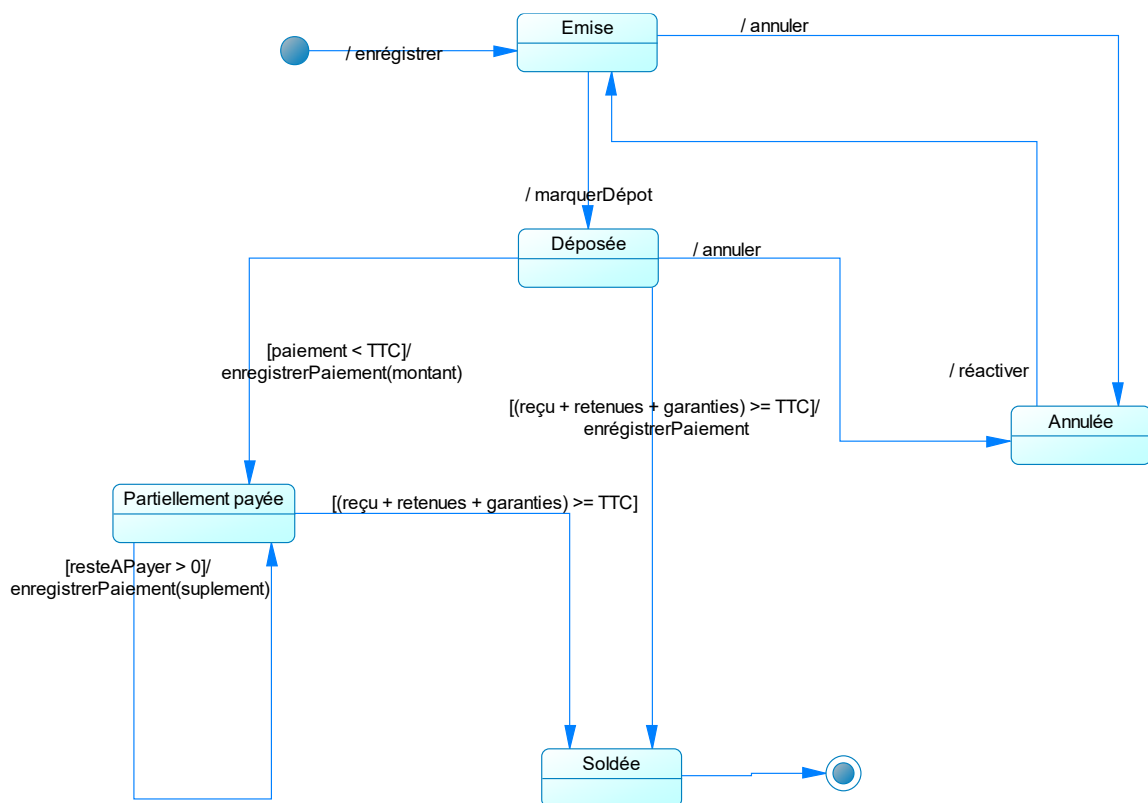


Figure 14 : Diagramme d'état-transition de l'objet FactureClient

Ce diagramme illustre les différentes étapes que peut traverser une facture une fois émise, ainsi que les transitions possibles entre ces états. Il montre comment une facture peut évoluer vers une situation déposée, partiellement payée, soldée ou annulée selon les actions effectuées dans le système.

□ Diagramme d'état d'une Garantie de bonne exécution (GBE)

La GBE suit un cycle de vie précis depuis son enregistrement jusqu'à son échéance. Comme l'illustre dans la *Figure 15*.

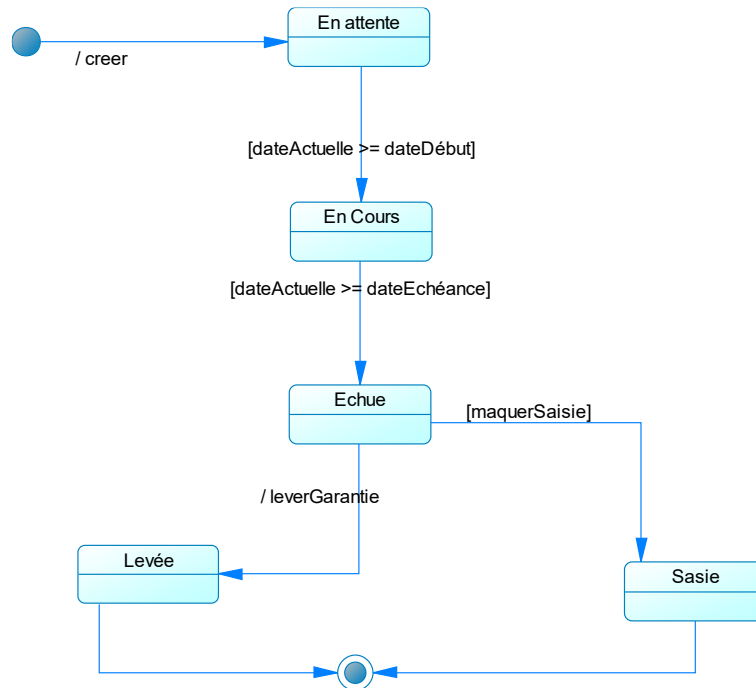


Figure 15 : Diagramme d'état-transition d'un objet GarantieBonneExecution

Comme l'illustre la Figure 15, la GBE évolue selon les étapes suivantes :

- « **En attente** » : la garantie est enregistrée dans le système mais pas encore entrée en vigueur.
- « **En cours** » : la garantie est activée et couvre la période de validité définie dans le contrat.
- « **Échue** » : la période de validité de la garantie arrive à son terme.
 - À ce stade, deux cas peuvent se produire :
 - « **Levée** » : si le marché a été correctement exécuté, la garantie est levée et que le montant bloqué est restitué à l'entreprise.
 - « **Saisie** » : si l'exécution du marché est jugée non conforme et que le client demande à la banque de conserver le montant garanti afin de couvrir les manquements.

Ce diagramme permet de visualiser clairement les différents états d'une GBE ainsi que les transitions possibles.

3. Diagramme de déploiement

Le diagramme de déploiement UML est un diagramme structurel qui a pour objectif de modéliser la configuration physique des artefacts d'un système. Il met en lumière les différents nœuds ainsi que les composants logiciels qui y sont déployés et leurs relations.

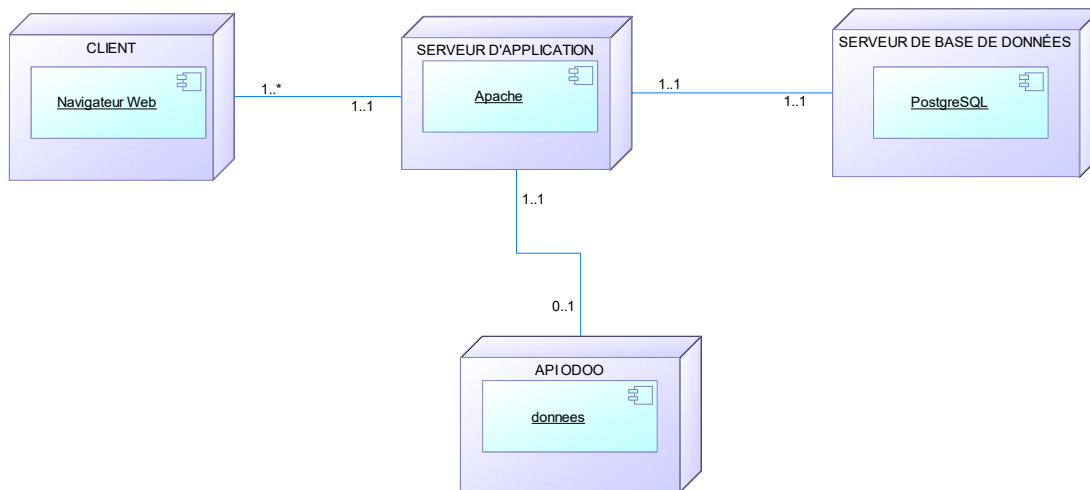


Figure 16 : Diagramme de déploiement

Comme l'illustre la *Figure 16*, le diagramme de déploiement de notre système comprend quatre (04) nœuds principaux interconnectés : le poste client (navigateur web), le serveur applicatif chargé de l'exécution du code métier, le serveur de base de données et l'API Odoo. Les utilisateurs, à travers leur navigateur web, accèdent à l'application et interagissent avec le serveur applicatif. Celui-ci assure le traitement des requêtes et gère la lecture, l'écriture et la mise à jour des données stockées dans le serveur de base de données. Il est également important de souligner que le serveur applicatif peut, si nécessaire, se connecter à l'instance Odoo existante via API par une communication JSON-RPC²³ afin récupérer certaines informations.

IV. Réalisation

Dans cette partie, nous présentons successivement les technologies utilisées, l'architecture logicielle, les interfaces IHM conçues, les mesures de sécurité ainsi que l'estimation des coûts liés à la mise en place du système.

²³ JSON-RPC : <https://www.jsonrpc.org/specification>

1. Présentation des outils développement

Dans le cadre de la réalisation de notre projet, nous avons utilisé plusieurs outils. Ces outils nous ont permis de concevoir, développer et mettre en œuvre la solution. Il s'agit notamment des langages de programmation, des plateformes de développement et des outils de conception.

a). Langages utilisés

Nous avons utilisé des langages de programmation, de balisage et de mise en forme pour le développement de notre système. Il s'agit des langages suivants :

- **PHP²⁴ 8.2** : langage de programmation côté serveur, utilisé pour développer la logique métier et interagir avec la base de données.
- **JavaScript²⁵ ES2022** : langage de programmation employé côté client pour ajouter de l'interactivité et dynamiser l'interface utilisateur.
- **TypeScript²⁶** : surensemble de JavaScript, utilisé pour introduire le typage statique et améliorer la fiabilité ainsi que la maintenabilité du code.
- **HTML5²⁷** (HyperText Markup Language) : langage de balisage, utilisé pour structurer les différentes pages Web.
- **CSS3²⁸** (Cascading Style Sheets) : langage de mise en forme utilisé pour définir la présentation et le design des pages web.

b). Frameworks de développement

Les plateformes de développement, ou *frameworks*, sont des environnements logiciels fournissant des composants réutilisables et des structures prédéfinies qui facilitent et accélèrent la création d'applications. Elles permettent de se concentrer sur les fonctionnalités métier tout en respectant de bonnes pratiques de développement. On peut scinder les plateformes de développement en deux (2) catégories ; on a des plateformes de développement back-end et des plateformes de développement front-end.

➤ Le choix du framework Backend

²⁴ <https://www.php.net/docs.php>

²⁵ <https://developer.mozilla.org/fr/docs/Web/JavaScript>

²⁶ <https://www.typescriptlang.org/docs>

²⁷ <https://www.w3.org/html>

²⁸ <https://developer.mozilla.org/fr/docs/Web/CSS>

Le backend constitue le cœur fonctionnel d’une application, responsable de la logique métier, du traitement des données, de la sécurité et de la communication avec la base de données. Avant de présenter le framework que nous avons retenu, le Tableau 15 propose une analyse comparative de différents frameworks backend couramment utilisés dans le développement d'applications web modernes.

Tableau 15 : Comparaison de quelques frameworks modernes

Critères	Laravel ²⁹ (PHP)	Django ³⁰ (Python)	Spring (Java)	Boot ³¹
Courbe d’apprentissage	Moyenne	Rapide	Élevée	
Rapidité de développement	Très rapide	Très Rapide	Moyenne	
Communauté	Très large	Large	Large	
Intégration API	Excellente	Excellente	Excellente	

Notre choix se porte sur le framework **Laravel** car il permet un développement rapide grâce à son écosystème complet, dispose d’une communauté active, et de plus, il correspond aux compétences techniques maîtrisées par notre groupe de projet, ce qui facilite sa mise en œuvre.

➤ Le choix du framework Frontend

Le frontend est la partie visible de l’application, chargée d’assurer l’interaction avec l’utilisateur. Le Tableau 16 présente une comparaison de frameworks frontend populaires avec des critères adaptés à notre contexte.

Tableau 16 : Comparaison de quelques frameworks frontend populaires

Critères	React.js ³²	Vue.js ³³	Angular ³⁴
Popularité	Très élevée	Élevée	Élevée
Simplicité	Bonne	Très bonne	Bonne

²⁹ <https://laravel.com/>

³⁰ <https://www.djangoproject.com/>

³¹ <https://spring.io/projects/spring-boot>

³² <https://react.dev>

³³ <https://vuejs.org/guide/introduction>

³⁴ <https://angular.dev/overview>

Performance	Excellente	Excellente	Bonne
Intégration Inertia.js³⁵	Oui	Oui	Non

Choix retenu : Nous retenons React.js, car il dispose d'une large communauté, d'un écosystème riche et d'une architecture basée sur les composants qui facilite la réutilisabilité du code. De plus, il s'intègre aisément avec Laravel grâce à Inertia.js, ce qui permet de développer une application moderne et fluide sans recourir à une API³⁶ distincte.

c). Outils de conception

Les outils de conception représentent ici l'ensemble des outils utilisés depuis la conception jusqu'à la réalisation de la plateforme. Ce sont :

- **PowerDesigner³⁷ 16.7** : outil de modélisation, utilisé pour concevoir les différents diagrammes UML ;
- **GanttProject³⁸** : logiciel de gestion de projet, utilisé pour élaborer le diagramme de Gantt et planifier les tâches du projet ;
- **pgAdmin 4³⁹** : outil d'administration, utilisé pour gérer et visualiser la base de données PostgreSQL ;
- **Git⁴⁰** : système de gestion de versions, utilisé pour gérer les versions du code source et suivre ses modifications ;
- **GitHub⁴¹** : plateforme d'hébergement, utilisée pour héberger le dépôt Git et centraliser le suivi collaboratif ;
- **Microsoft Word⁴²** : logiciel de traitement de texte, utilisé pour la rédaction des documents du projet ;
- **Microsoft PowerPoint⁴³** : logiciel de présentation, utilisé pour préparer les supports visuels des exposés et présentations ;

³⁵ <https://inertiajs.com> : bibliothèque qui permet d'intégrer facilement un framework frontend dans Laravel, sans recourir à une API.

³⁶ API : [Application Programming Interface](#)

³⁷ <https://www.sap.com/products/powerdesigner.html>

³⁸ <https://www.ganttproject.biz>

³⁹ <https://www.pgadmin.org>

⁴⁰ <https://git-scm.com/>

⁴¹ <https://github.com/>

⁴² <https://www.microsoft.com/office>

⁴³ <https://www.microsoft.com/office>

- **Visual Studio Code⁴⁴** : environnement de développement, utilisé pour écrire, organiser et maintenir le code de l'application ;

2. Infrastructure et l'environnement d'exécution

L'infrastructure d'exécution désigne l'environnement technique indispensable au fonctionnement de l'application. Elle regroupe les composants serveurs permettant d'héberger, d'exécuter et de diffuser l'application aux utilisateurs finaux. Dans le cas d'une application Laravel, cette infrastructure se compose principalement du serveur web, de l'environnement d'exécution PHP et du système de gestion de base de données. Ici nous présentons nos choix concernant le serveur web et le SGBD.

a). Choix du serveur web

Le serveur web représente le point d'entrée de l'application, responsable de la réception des requêtes HTTP, de leur traitement et de la transmission des réponses aux clients. Plusieurs solutions existent sur le marché. Nous proposons dans le Tableau 17 une analyse comparative des serveurs web les plus courants.

Tableau 17 : Comparaison de différents serveurs web

Serveur Web	HTTPS	Console d'admin		Coût	Plateformes
Nginx⁴⁵	Oui	Via plugin	Grande	Gratuit	Linux, *BSD, Windows, macOS
Apache HTTP Server⁴⁶	Oui	Via plugin	Très Grande	Gratuit	Linux, *BSD, Windows, macOS, Solaris
LiteSpeed / OpenLiteSpeed⁴⁷	Oui	Intégrée	Moyenne	OpenLiteSpeed gratuit / Enterprise payant	Linux, *BSD

⁴⁴ <https://code.visualstudio.com>

⁴⁵ Nginx : https://nginx.org/en/docs/httpngx_http_ssl_module.html

⁴⁶ Apache : <https://httpd.apache.org/docs/>

⁴⁷ OpenLiteSpeed : <https://docs.openlitespeed.org>

Internet Information Services (IIS)⁴⁸	Oui	Intégrée	Moyenne	Licence payante	Windows
---	-----	----------	---------	-----------------	---------

Choix retenu : Notre choix se porte sur **Apache HTTP Server**. Ce serveur Web offre une intégration éprouvée avec PHP via PHP-FPM⁴⁹, ce qui assure une compatibilité totale avec notre framework Laravel. Il bénéficie d'une très large communauté active, garantissant fiabilité, documentation et support. De plus, étant open-source et gratuit, il réduit les coûts et s'intègre facilement à l'infrastructure déjà en place dans l'entreprise, ce qui facilite sa mise en œuvre et sa maintenance.

b). Choix Système de Gestion de Base de données (SGBD)

Les SGBD se classent traditionnellement en deux grandes familles : les SGBD relationnels (SQL), où les données sont structurées en tables avec un schéma explicite et des contraintes d'intégrité, et les SGBD non relationnels (NoSQL)⁵⁰, plus flexibles, organisant les données en documents, clé-valeur, colonnes ou graphes. Le choix entre ces paradigmes, puis entre les solutions spécifiques, est fondamental et doit être guidé par la nature des données, les contraintes de performance et les besoins fonctionnels du projet.

Pour notre application, qui traite majoritairement des données structurées et nécessite des transactions ACID⁵¹ ainsi que des requêtes relationnelles, le modèle relationnel s'impose comme le choix le plus adapté pour garantir cohérence, intégrité et traçabilité. Nous menons une étude comparative des SGBD relationnels les plus répandus, en fonction de critères alignés sur nos besoins : licence, coût, compatibilité avec notre stack technique, performances, sécurité et évolutivité dans le Tableau 18 .

⁴⁸ IIS : <https://learn.microsoft.com/en-us/iis/>

⁴⁹ PHP-FPM (FastCGI Process Manager) : Gestionnaire de processus PHP

⁵⁰ <https://aws.amazon.com/compare/the-difference-between-relational-and-non-relational-databases>

⁵¹ Atomicité, Cohérence, Isolation, Durabilité

Tableau 18 : Comparaison de quelques systèmes de gestion de bases de données relationnels

Critères	PostgreSQL ⁵²	MySQL ⁵³ / MariaDB ⁵⁴	Microsoft SQL Server ⁵⁵	Oracle Database ⁵⁶
Licence	Open source	Open source (Community)	Propriétaire	Propriétaire
Coût	Gratuit, support optionnel	Gratuit ou payant selon édition	Gratuit en Express, coûts élevés en éditions supérieures	Très élevé (licences + support)
Compatibilité	Multi-plateformes ; utilisé nativement par Odoo ; bonne intégration avec Laravel/ORM	Multi-plateformes ; forte adoption web	Principalement Windows ; très intégrés à Microsoft (.NET, Power BI)	Fonctionne sur Linux/Windows, très orienté écosystème Oracle
Performance	Excellente en charges lourdes et transactions complexes	Très bonne pour applications web classiques	Excellente, optimisée pour environnement Microsoft	Excellente, adaptée aux grandes entreprises
Sécurité	Sécurité avancée (chiffrement, gestion fine des droits)	Sécurité standard (SSL/TLS)	Sécurité renforcée ; options avancées	Sécurité très robuste
Évolutivité	Très élevée (extensions, réplication native)	Moyenne (réplication, partitionnement)	Bonne mais dépend des éditions payantes	Très élevée mais coûteuse et complexe

Choix retenu : Au terme de cette comparaison, **PostgreSQL** apparaît comme le choix le plus adapté à notre projet. Il combine gratuité, robustesse et compatibilité avec l'environnement

⁵² <https://www.postgresql.org/docs>

⁵³ <https://dev.mysql.com/doc>

⁵⁴ <https://mariadb.com/docs>

⁵⁵ [MS SQL Server](#)

⁵⁶ [Oracle](#)

existant de l'entreprise, ce qui en fait une solution fiable et durable pour la mise en place de notre système.

3. Présentation de l'architecture logicielle

Il existe plusieurs patterns architecturaux⁵⁷ permettant d'organiser le développement logiciel, parmi lesquels MVP⁵⁸, MVVM⁵⁹ et MVC⁶⁰. Chacun propose une manière de structurer l'interaction entre les données, l'interface et la logique métier.

Dans le cadre de notre projet, nous retenons directement l'architecture MVC (Model–View–Controller), qui est nativement intégrée au framework Laravel. Ce paradigme de développement, largement utilisé dans les applications web, offre une séparation claire des responsabilités au sein de l'application en trois composants distincts :

□ Le Modèle (Model)

Il représente la couche métier de l'application. Il contient les données manipulées, les règles de gestion et la logique de persistance. Le modèle est indépendant de l'interface utilisateur.

□ La Vue (View)

Elle est en charge de l'affichage des données. Elle présente visuellement les informations du modèle, se met à jour lors des changements et capture les interactions de l'utilisateur qu'elle transmet au contrôleur.

□ Le Contrôleur (Controller)

Il assure l'interface entre la vue et le modèle. Il traite les actions de l'utilisateur, modifie le modèle si nécessaire et définit la vue à afficher en conséquence.

Ainsi, l'utilisateur interagit avec la vue, qui transmet ses actions au contrôleur. Ce dernier agit sur le modèle, puis la vue est actualisée avec les nouvelles données.

⁵⁷ https://fr.wikipedia.org/wiki/Patron_de_conception

⁵⁸ MVP : Model–View–Presenter

⁵⁹ MVVM : Model–View–ViewModel

⁶⁰ MVC : Model–View–Controller

La Figure 17 présente une illustration du modèle MVC.

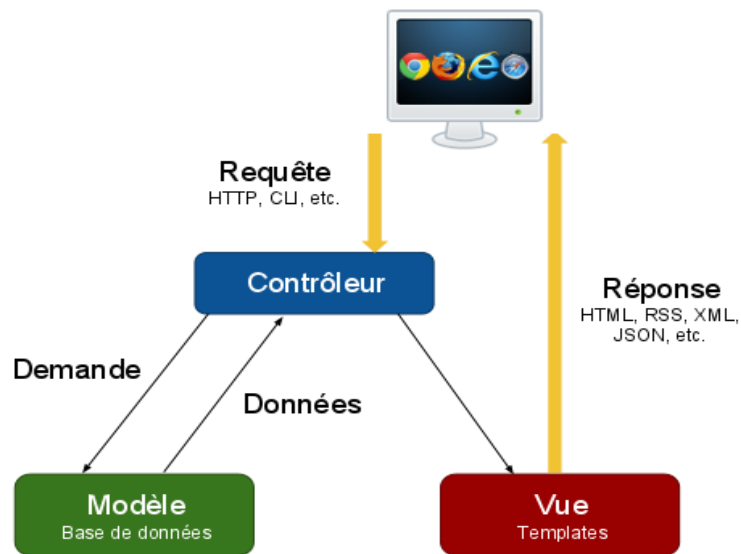


Figure 17: Schéma d'illustration de l'architecture MVC⁶¹

L'architecture MVC présente plusieurs avantages, parmi lesquels :

- Séparation des préoccupations : chaque couche a un rôle bien défini, ce qui rend le code plus clair et maintenable ;
- Réutilisation du modèle : les mêmes données peuvent être affichées dans différentes interfaces ;
- Testabilité : chaque composant peut être testé de manière indépendante.

⁶¹ Source : : <https://raphaelbellon.wordpress.com/wp-content/uploads/2017/03/mvc.png>

4. Quelques interfaces (IHM) de notre application



Figure 18 : Écran page de destination

Cet écran présente la page d'accueil de l'application. Il sert de point d'entrée pour guider l'utilisateur vers la page de connexion afin d'accéder au système.

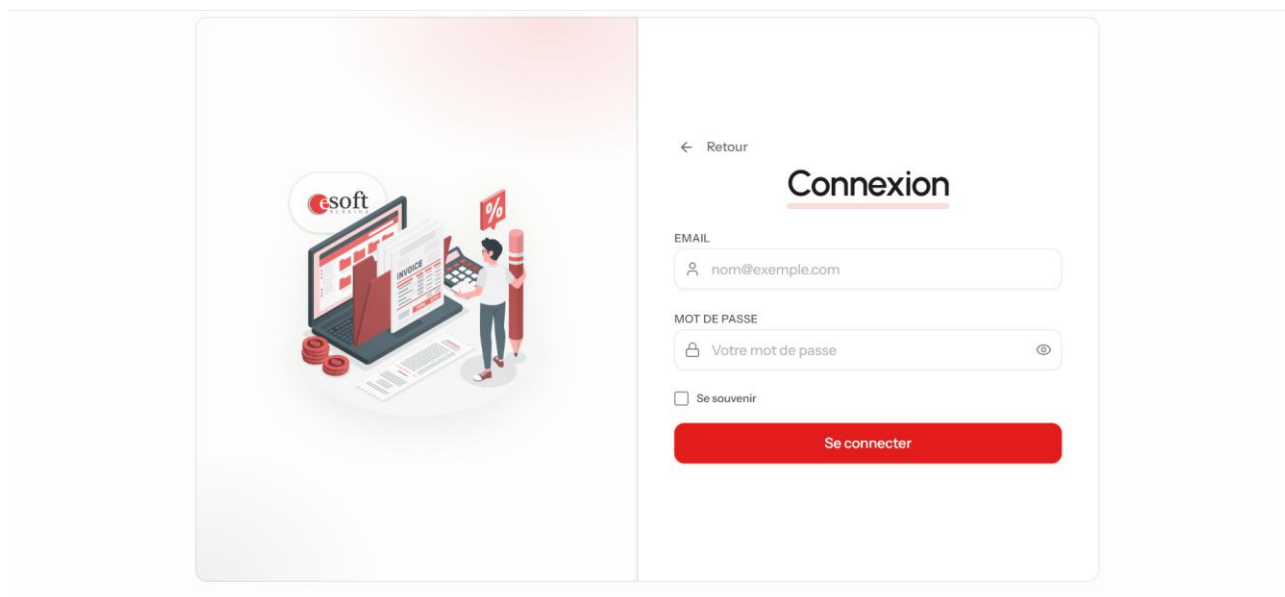


Figure 19 : Écran de connexion

Cet écran affiche le formulaire permettant à l'utilisateur de saisir ses identifiants. Il constitue l'étape préalable pour accéder aux différentes fonctionnalités du système.

Figure 20 : Écran d'enregistrement d'une facture client

Cet écran permet de saisir les informations nécessaires à l'enregistrement d'une nouvelle facture client. Il offre un formulaire structuré pour renseigner les données essentielles avant la validation.

Référence & Client	Bon de commande & Intitulé	Date de dépôt & Échéance	Montant TTC	Statut	Actions
ESB/2025/00011 10% UNFPA	Acquisition d'équipements informatiques pour la CNDH N° BC : 70167445	25 févr. 2025 25 févr. 2025	1 917 500 CFA	Payée 1 755 000 CFA	
TEST/2025/00006 10% nabivo	Ordinateur ASUS GAMING N° BC : 10000397	08 oct. 2025 07 nov. 2025	11 800 000 CFA	Payée 10 800 000 CFA	
TEST/2025/00005 10% nabivo	Achat de Télé Samsung N° BC : 10000396	08 oct. 2025 07 nov. 2025	354 000 CFA	Partiel 324 000 CFA	
TEST/2025/00004 10% nabivo	Développement d'un logiciel de factures N° BC : 10000395	08 oct. 2025 07 nov. 2025	11 800 000 CFA	Payée 11 800 000 CFA	
TEST/2025/00003 4% nabivo	Achat d'un ordinateur HP Core i9 N° BC : 000543	08 oct. 2025 07 nov. 2025	354 000 CFA	Partiel 10 000 CFA Reste: 332 000 CFA	
TEST/2025/00002 10% OUEDRAOGO ABDOL RAZACK	Acquisition de matériel informatique N° BC : 000542	06 oct. 2025 05 nov. 2025	70 800 CFA	Impayée	
ESB/2025/00060 10% MICOMAHOMBO Gold Dentiste	Acquisition d'un écran HP 5325F N° BC : 111111	09 sept. 2025	236 000 CFA	Impayée	

Figure 21 : Écran de quelques factures clients enregistrées

Cet écran présente l'ensemble des factures enregistrées avec leurs informations principales et leurs statuts. Il permet à l'utilisateur de consulter, filtrer ou rechercher rapidement une facture spécifique.

5. Tests

Les tests constituent une étape essentielle dans le cycle de vie d'un projet logiciel. Ils permettent de vérifier la conformité du système aux spécifications fonctionnelles, de détecter d'éventuelles anomalies et d'assurer une qualité suffisante avant la mise en production. En génie logiciel, on distingue plusieurs catégories de tests :

- tests unitaires : vérifient les plus petites unités de code (méthodes, classes) pour valider leur exactitude ;
- tests fonctionnels (ou d'intégration) : contrôlent que les fonctionnalités définies dans les spécifications sont correctement implémentées et que les modules interagissent bien entre eux ;
- tests structurels : portent sur la structure interne du code et visent une couverture maximale des instructions, conditions et chemins ;
- tests de performance et de charge : évaluent la rapidité et la robustesse du système sous contrainte ;
- tests de sécurité : mesurent la résistance de l'application face aux intrusions ou aux accès non autorisés ;
- tests d'acceptation (ou de recette) : réalisés en fin de projet, ils confirment que la solution répond bien aux attentes des utilisateurs.

Toutefois, dans le cadre de ce rapport, nous nous limitons à présenter quelques cas représentatifs de tests unitaires et fonctionnels.

➤ Les outils et environnement de test

Pour la mise en place des tests, nous avons exploité l'infrastructure intégrée de Laravel :

- cadre de test : *PHPUnit*, livré par défaut avec Laravel, a été utilisé pour exécuter les tests unitaires et fonctionnels ;
- génération de données : *Faker* et les *Model Factories* ont servi à créer des jeux de données réalistes ;
- base de données de test : un fichier `.env.testing` a été configuré pour utiliser *SQLite en mémoire*, permettant d'exécuter les tests sans dépendre d'une base réelle ;
- traits Laravel : *RefreshDatabase* et *DatabaseMigrations* ont permis d'assurer l'isolation entre chaque test en réinitialisant les tables à chaque exécution ;

- doubles de test : nous avons utilisé `Http::fake()`, `Queue::fake()` et `Storage::fake()` afin de simuler les dépendances externes et éviter les effets indésirables ;
- exécution des tests : réalisée via la commande *php artisan test* qui fournit un retour en temps réel sur le nombre de tests exécutés, réussis ou échoués.

➤ L'organisation des tests

Laravel propose par défaut une organisation des tests en deux suites principales :

- unit tests (tests unitaires) : placés dans le répertoire `tests/Unit`, ils valident les règles de gestion et le comportement des méthodes ;
- feature tests (test fonctionnels) : localisés dans `tests/Feature`, ils couvrent les scénarios utilisateurs via les routes HTTP et les contrôleurs.

Cette organisation, propre à Laravel, facilite la séparation entre la validation des règles métier et la vérification des scénarios utilisateurs complets.

➤ Les jeux de tests

Les tableaux (Tableau 19 et

Ce tableau présente quelques cas de tests unitaires appliqués aux fonctions clés du système. Il permet d'illustrer le comportement attendu de certaines opérations telles que le calcul du reste à payer, la prise en compte des retenues ou la mise à jour du statut d'une facture.

Tableau 20) illustrent quelques cas de tests unitaires et fonctionnels que nous avons mis en œuvre.

Tableau 19 : Jeu de test unitaire avant le test

ID	Description	Entrée	Sortie attendue	Statut
TU001	Calcul du reste à payer sur une facture	Facture TTC=200 000 ; aucun paiement	resteAPayer() → 200 000	
TU002	Retenue et montant perçu	Montant=200 000 ; taux=5 %	Retenue=10 000 ; Net=190 000	
TU003	Transition de statut d'une facture.	Facture TTC=200 000 Paiements 50 000 puis 150 000	Après 50 000 → PAYEE_PARTIEL ; après 150 000 → SOLDEE	
TU004	Échéance et rappel d'une GBE	Début=01/09/2026 ; durée=12 mois	Échéance=01/09/2026 ; rappelJMoins(30) = true	

Ce tableau présente quelques cas de tests unitaires appliqués aux fonctions clés du système. Il permet d'illustrer le comportement attendu de certaines opérations telles que le calcul du reste à payer, la prise en compte des retenues ou la mise à jour du statut d'une facture.

Tableau 20 : Jeu de test fonctionnel avant le test

ID du Test Fonctionnel	Description	Entrée	Sortie attendu	Statut
TF001	Enregistrement d'une facture client (avec pré-remplissage Odoo)	Données valides {client_id, ref unique, date + données trouvées sur odoo}.	Code 302 (redirection facture-client/index); facture créée en BDD avec champs.	
	Enregistrement d'une facture client	Données invalides (ex. ref vide/duplicata, montant ≤ 0)	Code 422 (erreurs de validation) et code 302 (retour form) ; aucune facture créée	
TF002	Enregistrement d'une facture fournisseur	Données valides {fournisseur_id, ref unique, date} ;	Code 302 ; facture fournisseur créée en BDD.	
	Enregistrement d'une facture fournisseur	Données invalides (montant ≤ 0 , fournisseur_id absent)	Code 422 (erreurs de validation) et code 302 (retour form) ; aucune création	
TF003	Enregistrement d'un règlement de facture client	Facture client existante non soldée ; {facture_id, ref, mode, montant > 0 , date}	Code 302 ReglementFacture créé en BDD ; solde et statut de la facture mis à jour ;	
	Enregistrement d'un règlement de facture client	ref manquante, montant ≤ 0 , facture_id inexistant	Code 422 (validation) ou 404 (facture introuvable) ; aucun enregistrement	

Ce tableau regroupe plusieurs tests fonctionnels destinés à vérifier le bon déroulement des principales actions métiers, comme l'enregistrement des factures et des paiements. Il met en fonction de différentes situations, avec ou sans erreurs de validation

➤ L'exécution des tests

Chaque cas de test a été implémenté dans une classe PHP dédiée, en tenant compte des entrées et des sorties attendues. Les figures Figure 22 et Figure 23 présentent respectivement les exécutions des tests unitaires et des tests fonctionnels définies.

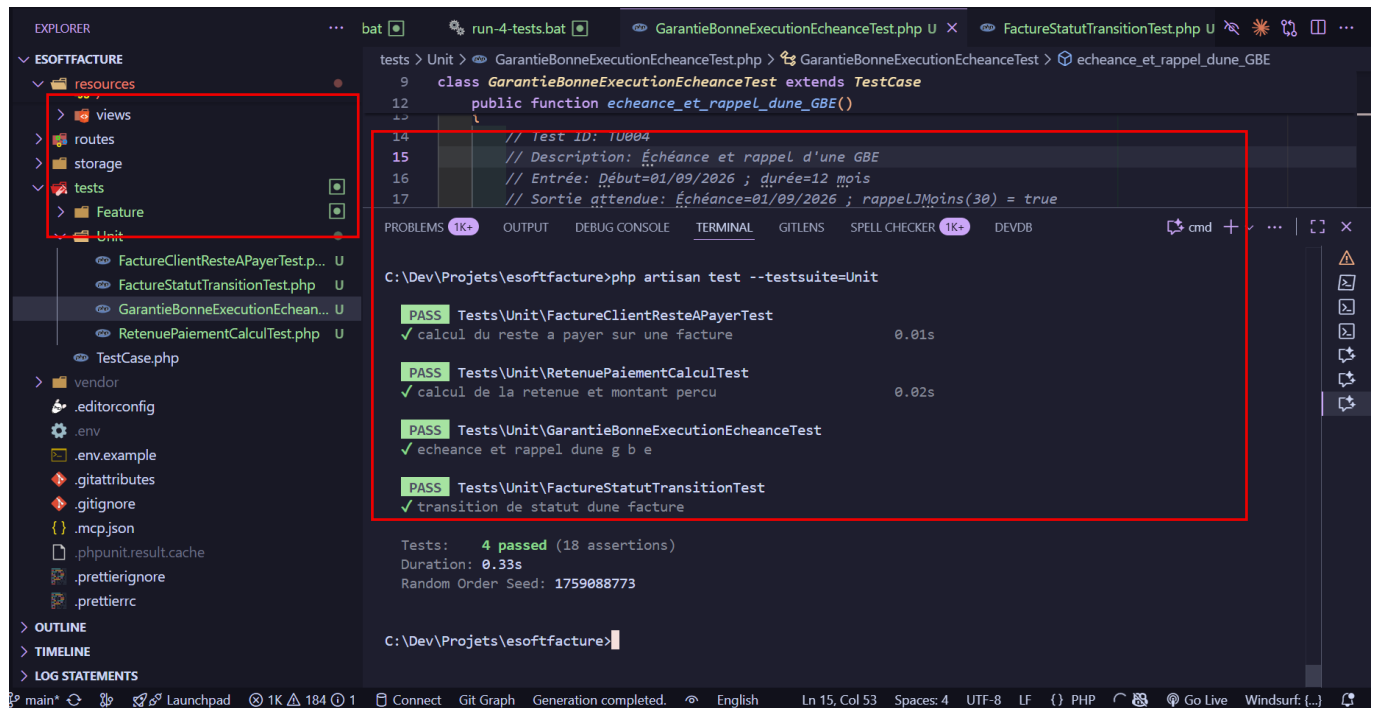


Figure 22 : Rapport de quelques tests cas de test unitaires

Cette exécution montre que les quatre tests unitaires définis ont été validés avec succès en 0,33 secondes. Aucun échec, erreur ou test ignoré n'a été constaté, ce qui confirme la bonne implémentation des règles métiers essentielles.

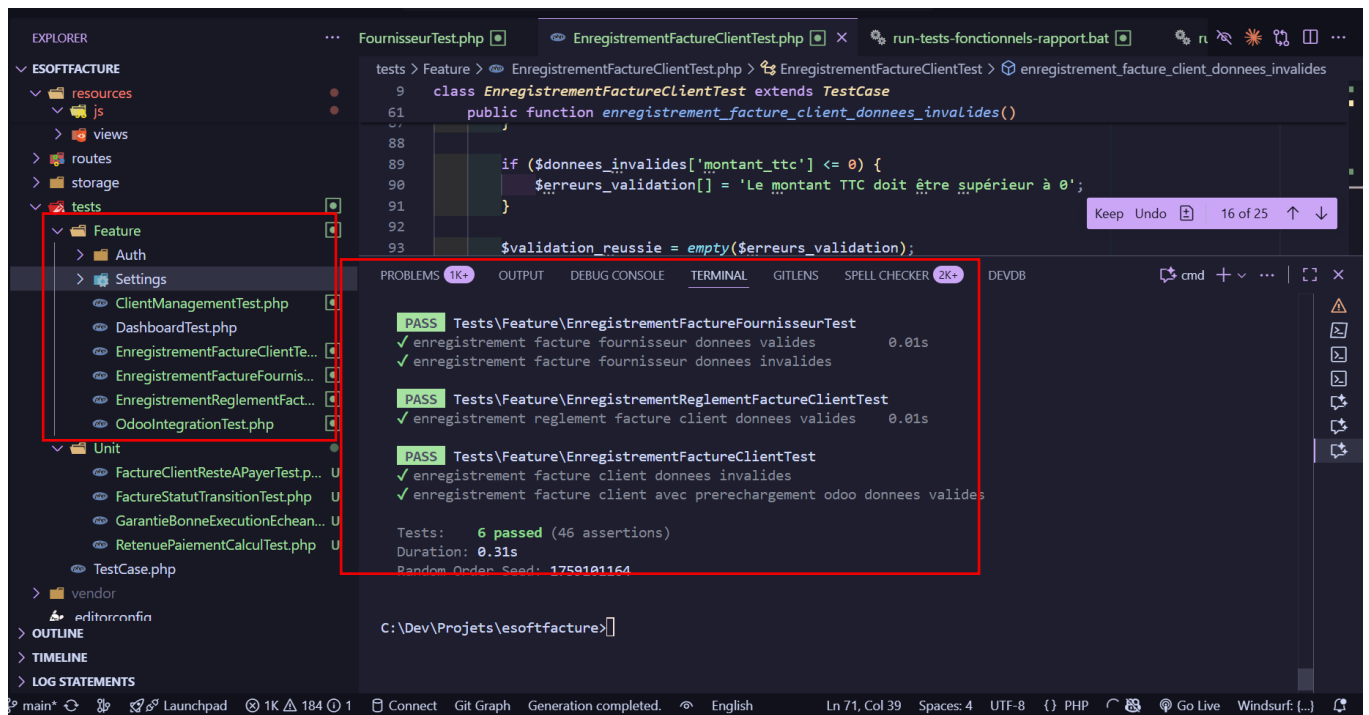


Figure 23 : : Rapport de quelques tests cas de test fonctionnels

Les tests fonctionnels ont également été validés avec succès. Le rapport indique que 6 tests ont été passés couvrant plusieurs scénarios avec un temps d'exécution de 0,31 secondes. Là encore, aucun échec n'a été enregistré.

➤ Les résultats et interprétations

Tableau 21 : Jeu de test unitaire après le test

ID	Description	Entrée	Sortie attendu	Statut
TU001	Calcul du reste à payer sur une facture	Facture TTC=200 000 ; aucun paiement	resteAPayer() → 200 000	PASS
TU002	Retenue et montant perçu	Montant=200 000 ; taux=5 %	Retenue=10 000 ; Net=190 000	PASS
TU003	Transition de statut d'une facture.	Facture TTC=200 000 Paiements 50 000 puis 150 000	Après 50 000 → PAYEE_PARTIEL ; après 150 000 → SOLDEE	PASS
TU004	Échéance et rappel d'une GBE	Début=01/09/2025 ; durée=12 mois	Échéance=01/09/2026 ; rappelJMoins(30) = true	PASS

Tableau 22: Jeu de test fonctionnel après le test

ID du Test Fonctionnel	Description	Entrée	Sortie attendu	Statut
TF001	Enregistrement d'une facture client (avec pré-remplissage Odoo)	Données valides {client_id, ref unique, date + données trouvées par odoo}.	Code 302 (redirection facture-client/index); facture créée en BDD avec champs.	PASS
	Enregistrement d'une facture client	Données invalides (ex. ref vide/duplicata, montant ≤ 0 , client_id manquant)	Code 422 (erreurs de validation) et code 302 (retour form) ; aucune facture créée	PASS
TF002	Enregistrement d'une facture fournisseur	Données valides {fournisseur_id, ref unique, date} ;	Code 302 ; facture fournisseur créée en BDD.	PASS
	Enregistrement d'une facture fournisseur	Données invalides (montant ≤ 0 , fournisseur_id absent)	Code 422 (erreurs de validation) et code 302 (retour au formulaire) ; aucune création	PASS
TF003	Enregistrement d'un règlement de facture client	Facture client existante non soldée ; {facture_id, ref, mode, montant > 0, date}	Code 302 ReglementFacture créé en BDD ; solde et statut de la facture mis à jour ; confirmation affichée	PASS
	Enregistrement d'un règlement de facture client	ref manquante, montant ≤ 0 , facture_id inexistant	Code 422 (validation) ou 404 (facture introuvable) ; aucun enregistrement	PASS

Au total, l'ensemble des tests unitaires et fonctionnels réalisés se sont soldés par des statuts *PASS*. Ces résultats satisfaisants traduisent la robustesse et la cohérence du système. Bien que le jeu de tests soit limité, il couvre les fonctionnalités importantes et fournit une base fiable pour une validation plus large lors des futures phases, et ouvre la voie à des campagnes de tests plus complètes, notamment en matière de performance et de sécurité.

6. Mesures de sécurité

La mise en place d'un système comme le nôtre implique de traiter des données particulièrement sensibles, telles que les montants financiers et les références de paiements ainsi que les

informations relatives aux clients et fournisseurs. Il est donc indispensable de définir et de mettre en œuvre un ensemble de mesures robustes de sécurité afin de garantir la confidentialité, l'intégrité et la disponibilité de ces données. Dans cette partie, nous détaillons les mesures que nous avons implémentées pendant le développement, nos recommandations en matière de sécurité pour l'exploitation et la mise en production, ainsi que des recommandations organisationnelles et humaines.

➤ **Les mesures implémentées durant le développement**

Dès la phase de conception, l'accent a été mis sur la protection des accès et des données :

- l'authentification sécurisée : chaque utilisateur doit disposer d'un identifiant unique et d'un mot de passe, qui est chiffré dans la base de données pour prévenir toute compromission ;
- la gestion fine des droits d'accès : un système de rôles et de permissions est implémenté. Cela garantit que chaque utilisateur n'accède qu'aux fonctionnalités et aux informations strictement nécessaires à ses responsabilités ;
- la validation des données : tous les formulaires de l'application intègrent des contrôles de saisie systématiques pour éviter les erreurs et réduire les risques d'injection de données malveillantes.

➤ **Les recommandations pour l'exploitation et la mise en production**

Lors de la mise en production et de l'utilisation du système, il est nécessaire de prévoir des mécanismes complémentaires afin d'assurer une exploitation sécurisée :

- un hébergement sécurisé : bien que l'application soit prévue pour un déploiement sur un serveur VPS⁶² chez OVH⁶³, il est recommandé de s'assurer en permanence que l'hébergeur respecte des normes de sécurité élevées, et d'envisager cette exigence pour tout futur changement.
- un plan de sauvegarde : nous préconisons de mettre en place un plan de sauvegarde automatisé et régulier (journalier, hebdomadaire, mensuel) avec une politique de conservation claire pour limiter les pertes de données en cas d'incident.
- la haute disponibilité : pour garantir la continuité du service, nous recommandons la mise en place d'un mécanisme de réplication des données sur un serveur de secours.

➤ **Les recommandations organisationnelles et humaines**

⁶² VPS (Virtual Private Server) : serveur privé virtuel

⁶³ OVH : <https://www.ovhcloud.com/fr/>

La sécurité ne se limite pas aux aspects techniques. Elle dépend également des comportements et des pratiques adoptés par les utilisateurs, et doit être renforcée par de bonnes pratiques organisationnelles. Il est ainsi recommandé à l'entreprise de :

- sensibiliser les utilisateurs : promouvoir les bonnes pratiques telles que la non-divulgaration des identifiants, le verrouillage de la session en cas d'absence et l'utilisation de mots de passe complexes ;
- maintenir une hygiène informatique : s'assurer que les postes de travail sont équipés d'un antivirus à jour et encourager les utilisateurs à éviter les sites web non sécurisés.

7. Estimation des coûts du projet

L'estimation des coûts constitue une étape fondamentale dans la conception d'un logiciel, car elle permet d'anticiper les ressources nécessaires à la réalisation du projet. Plusieurs méthodes d'estimation existent. Afin d'évaluer notre logiciel, nous présentons dans le Tableau 23 une synthèse des approches les plus répandues.

Tableau 23 : Comparaison des méthodes d'estimation de coût

Méthode	Principe	Avantages	Inconvénients
Jugement d'experts ⁶⁴	Estimation basée sur l'opinion de personnes expérimentées, en se référant à leur connaissance des coûts passés.	– S'appuie sur l'expérience réelle des développeurs et gestionnaires de projet. – Rapide pour obtenir une estimation approximative.	– Subjectif et dépend de l'expertise disponible. – Difficulté à justifier l'estimation pour un audit. – Risque de biais et d'erreur humaine.
Analogie (comparaison) ⁶⁵	Estimation basée sur la comparaison avec des projets similaires déjà réalisés.	– Basé sur des faits réels, donc potentiellement plus fiable. – Permet de capitaliser sur l'expérience passée.	– Moins fiable si les projets comparés sont trop différents. – Ne prend pas en compte les spécificités nouvelles du projet.

⁶⁴ <https://asana.com/fr/resources/expert-judgment>

⁶⁵ <https://clickup.com/fr-FR/blog/169879/l'estimation-par-analogie-dans-la-gestion-de-projet>

Bottom-up (décomposition)⁶⁶	Décomposition du projet en tâches/modules. Estimation individuelle du coût de chaque tâche, puis agrégation pour obtenir le coût total.	– Permet une estimation très précise du coût global. – Facilite l'identification des postes de dépense importants. – Transparente et justifiable pour la planification budgétaire.	– Peut être longue à mettre en œuvre pour de nombreux modules. – Risque de sous-estimer certains coûts si des tâches sont oubliées.
Paramétrique (ex : COCOMO)	Estimation basée sur des formules ou modèles mathématiques liés à la taille, complexité ou nombre de fonctionnalités du logiciel.	– Méthode objective et reproductible. – Permet une estimation rapide si les paramètres sont connus. – Réduit la subjectivité par rapport à l'expert ou analogie.	– Nécessite des données précises sur taille et complexité. – Peu flexible face aux spécificités du projet. – Peut être complexe à appliquer sans expertise technique.

Pour notre projet, nous retenons la **méthode ascendante (Bottom-Up)**. Elle consiste à décomposer le projet en phases et en modules, à estimer l'effort requis pour chaque tâche en jours-hommes, puis à convertir cet effort en coût en appliquant un taux journalier moyen. Enfin, une marge est intégrée afin de prendre en compte les imprévus. Notre choix se justifie par plusieurs éléments :

- la décomposition en tâches élémentaires permet une estimation plus fine et plus réaliste ;
- les estimations peuvent être ajustées et recalibrées facilement en fonction de l'avancement ;
- la méthode offre une vision claire des dépendances entre les différentes activités ;

⁶⁶<https://www.pacte.ch/gestion-projet/laproche-destination-bottom-up-un-guide-complet-pour-une-planification-precise/>

- la transparence du processus garantit une meilleure traçabilité et une justification solide des estimations auprès des parties prenantes.

➤ **La détermination du salaire journalier moyen**

En considérant que le salaire mensuel moyen d'un développeur web se situe entre 250 000 et 500 000 FCFA.

Pour obtenir une valeur de référence, nous retenons la moyenne :

$$\text{Salaire moyen mensuel} = \frac{250\,000 + 500\,000}{2} = 375\,000 \text{ FCFA/mois}$$

Et en considérant 22 jours ouvrés par mois, le taux moyen journalier moyen est :

$$\text{Taux Moyen Journalier} = \frac{375\,000}{22} = 17\,045 \text{ FCFA/jour}$$

➤ **Le cout de l'implémentation**

Le coût d'implémentation correspond à la phase de codage et de construction du logiciel. Pour son estimation, nous nous appuyons sur les cas d'utilisation identifiés lors de l'analyse, que nous regroupons en modules fonctionnels.

Chaque module se voit attribuer une durée estimée en jours-hommes, puis convertie en coût grâce au taux journalier moyen établi précédemment. Les résultats détaillés de ce calcul sont présentés dans le tableau ci-dessous.

Tableau 24 : Estimation du cout de développement

Module fonctionnel	Durée (jours- hommes)	Coût estimé (FCFA)
Authentification	4	68 180
Gestion des utilisateurs	5	85 225
Configuration système	4	68 180
Gestion des tiers (clients & fournisseurs)	10	170 450
Gestion des garanties de bonne exécution	6	102 270
Gestion des factures (clients & fournisseurs)	12	204 540
Gestion des règlements (clients & fournisseurs)	12	204 540
Consultation factures et tableaux de bord	5	85 225
Rapports et exports	7	119 315

Total Implémentation	65	1 107 925
----------------------	-----------	------------------

➤ **Le coût du développement (toutes phases hors matériel)**

Au-delà de l'implémentation, le coût du développement englobe l'ensemble des phases du cycle de vie, notamment l'analyse, la conception, l'implémentation, les tests, le déploiement et la formation des utilisateurs. Nous conduisons toujours cette estimation conformément à la méthode retenue, et les résultats détaillés sont présentés dans le tableau.

Phase	Durée (jours-hommes)	Coût estimé (FCFA)
Analyse & conception	20	340 900
Implémentation	65	1 107 925
Tests & intégration	12	204 540
Déploiement & documentation	6	102 270
Formation utilisateurs	4	68 180
Total (hors marge)	107	1 823 815

Afin de couvrir les incertitudes comme retards, bugs et ajustements de spécifications, nous appliquons une marge de 20 % :

$$\text{Marge (20\%)} = 1\,823\,815 \times 0,20 = \mathbf{364\,763\,FCFA}$$

$$\text{Coût total développement} = 1\,823\,815 + 364\,763 = \mathbf{2\,188\,578\,FCFA}$$

➤ **Coût global du projet**

En somme, en ajoutant les coûts matériels et logiciels nécessaires, nous obtenons le coût global du projet détaillé dans le Tableau 25 .

Tableau 25 : Coût global de réalisation du projet

Désignation	Quantité	Coût unitaire (FCFA)	Coût total (FCFA)
Ordinateur portable	1	300 000	300 000
PostgreSQL	1	Licence gratuite	0

Visual Studio Code	1	Licence gratuite	0
Développement	-	-	2 188 578
Total projet			2 488 578

Au terme de ce chapitre, nous avons présenté la problématique, les besoins réels de l'entreprise, ainsi que les étapes d'analyse, de conception et de réalisation de la solution numérique proposée, tout en justifiant les outils et choix technologiques retenus à chaque niveau. Le chapitre suivant présente le bilan global du stage, mettant en avant son déroulement, les activités menées et les compétences acquises.

CHAPITRE III : Bilan du stage.

Dans ce chapitre, nous dressons le bilan de notre stage au sein de ESOF BURKINA. Nous présentons d'abord le déroulement du stage et les principales activités menées, puis les connaissances et compétences acquises, avant de formuler des observations et des suggestions.

I. Déroulement du stage et les activités menées

Notre stage s'est déroulé du 08 juillet 2025 au 08 octobre 2025 au sein de ESOF BURKINA. D'une durée de trois (03) mois, il a été consacré à l'étude et à la mise en œuvre du thème « CONCEPTION ET REALISATION D'UN SYSTEME DIGITAL DE SUIVI DES FACTURES ET DES COMMANDES DANS UNE PME : CAS DE ESOF BURKINA SARL. », conformément au planning de réalisation établi.

1. Activités menées

Au cours de cette période, nous avons participé à des activités directement liées au projet et à la vie professionnelle de l'entreprise. Sur le plan technique, nous avons notamment :

- analysé les besoins de l'entreprise en matière de gestion des factures;
- conçu l'architecture de la plateforme ;
- développé les principales fonctionnalités ;
- réalisé les tests fonctionnels et préparé le déploiement de la solution.

En parallèle, nous avons pris part à plusieurs rencontres qui nous ont permis de comprendre le déroulement des projets en contexte institutionnel :

- une rencontre de cadrage pour la mise en place de la plateforme de suivi du Plan de Travail Annuel (PTA) au profit du Ministère de l'Économie et des Finances ;
- une présentation de la plateforme WARIKALAN (ANPFI, agréée par le MEF), dédiée à l'éducation financière ;
- des séances techniques consacrées à l'architecture physique de la base de données de la plateforme PTA ;
- une séance de restitution de l'étude diagnostique pour la validation des besoins de la plateforme PTA.

Ces activités ont contribué au renforcement de nos compétences techniques et à une meilleure compréhension des pratiques de gestion de projet en milieu institutionnel.

2. Connaissances acquises et compétences développées

Ce stage nous a été bénéfique à plusieurs niveaux. Il nous a permis de :

- mettre en pratique les connaissances académiques acquises durant nos trois (03) années de formation à l'IBAM ;
- renforcer nos compétences en développement d'applications web ;
- améliorer nos aptitudes de communication à travers les échanges professionnels et l'utilisation d'outils collaboratifs ;
- nous imprégner des réalités de la vie professionnelle ;
- découvrir et comprendre les principales fonctionnalités de l'ERP **Odoo** ;
- approfondir notre connaissance du fonctionnement administratif, des marchés publics, des appels d'offres, des offres techniques ainsi que du cycle de facturation.

II. Observations et suggestions

ESOFT BURKINA se distingue par sa rigueur dans le travail et son respect des délais. L'accueil réservé aux stagiaires ainsi que l'encadrement dont nous avons bénéficié favorisent une bonne intégration et témoignent d'une volonté de valoriser les jeunes professionnels. De plus, les allocations accordées constituent une marque de considération appréciable, particulièrement dans le contexte d'une PME. Toutefois, pour renforcer davantage son efficacité, nous suggérons la création d'une équipe interne de développement permanente, afin d'assurer une meilleure continuité des projets et de capitaliser sur les acquis techniques au sein de l'entreprise.

Ce chapitre a permis de présenter le déroulement du stage, les activités réalisées, les compétences développées ainsi que nos observations et suggestions pour l'entreprise. Il constitue une transition vers la conclusion générale, que nous exposons dans la suite de ce rapport.

CONCLUSION GÉNÉRALE

Au terme de ce stage réalisé au sein de l'entreprise ESOFT BURKINA SARL, nous avons procédé à une analyse approfondie du processus de gestion des factures et des commandes, afin d'en identifier les limites et de proposer une solution numérique adaptée. Ce travail nous a permis de mettre en application les connaissances acquises durant notre formation en MIAGE, tout en développant de nouvelles compétences techniques et méthodologiques dans un environnement professionnel réel.

En nous appuyant sur la démarche 2TUP, nous avons conçu et réalisé un système de suivi intégré des factures clients et fournisseurs, incluant la gestion des paiements, des retenues et des garanties de bonne exécution. Cette solution vise à améliorer la traçabilité des opérations, à réduire les risques d'erreurs et à fournir à l'entreprise un outil fiable d'aide à la décision financière.

Au-delà de l'aspect technique, cette expérience a été une véritable immersion dans la vie professionnelle. Elle nous a permis de mieux comprendre les réalités organisationnelles d'une PME, de renforcer notre autonomie, notre rigueur, et de mesurer l'importance de la communication avec les différents acteurs d'un projet.

Toutefois, ce travail ne constitue qu'une première étape. Des perspectives d'amélioration subsistent, notamment l'autonomisation complète du système, afin qu'il puisse couvrir l'ensemble du cycle complet de gestion des commandes et des factures. D'autres évolutions sont également prévues telles que la mise en place de fonctionnalités avancées de reporting, la prise en compte d'indicateurs de performance financière ainsi que le développement d'une application mobile. Ces prolongements permettront d'accroître encore davantage la valeur ajoutée du système pour l'entreprise.

En définitive, ce stage a été une expérience formatrice et déterminante, marquant une étape clé dans notre parcours académique et professionnel. Il a consolidé nos acquis tout en élargissant nos perspectives dans le domaine des systèmes d'information et de la transformation digitale des entreprises.

BIBLIOGRAPHIE ET WEBOGRAPHIE

1. Bibliographie

- ❖ [1] **TOE Adolphe Césaire (2022)**, « Conception et réalisation d'un logiciel de gestion de facture fournisseurs » Rapport de fin de cycle de Licence : MIAGE. Ouagadougou : Institut Burkinabè des Arts et Métiers (IBAM), 56p ;
- ❖ [2] **SAWADODO Rimalguedo Rahimata (2023)**, « Conception et réalisation d'une plateforme de gestion intégrée et de suivi des startups incubées à SIRA LAB » Rapport de fin de cycle de Licence : MIAGE. Ouagadougou : Institut Burkinabè des Arts et Métiers (IBAM), 58p ;
- ❖ [3] **SAWADOGO Patounizambo Yves (2024)**, « ANALYSE ET CONCEPTION D'UNE PLATEFORME INTEGRÉE DE REGISTRE ELECTRONIQUE DE VACCINATION » Rapport de fin de cycle de Licence : MIAGE. Ouagadougou : Institut Burkinabè des Arts et Métiers (IBAM), 80p ;
- ❖ [4] **OUEDRAOGO P. David (2021)**, « Mise en place d'un système de communication base sur les sms pour les professionnels et les particuliers » Rapport de fin de cycle de Licence : MIAGE. Ouagadougou : Institut Burkinabè des Arts et Métiers (IBAM), 64p ;
- ❖ [5] **GUIRE Ismaël Faïssal (2023)**, « Automatisation des processus d'approbation dans l'administration des systèmes informatiques au sein de TICANALYSE » Rapport de fin de cycle de Licence : MIAGE. Ouagadougou : Institut Burkinabè des Arts et Métiers (IBAM), 84p ;
- ❖ [6] **KAMBOU Yéri Sheila Mariane (2023)**, « MISE EN PLACE D'UN SYSTÈME DE GESTION DE MISSIONS : CAS DE A2SYS CONSULTING » Rapport de fin de cycle de Licence : MIAGE. Ouagadougou : Institut Burkinabè des Arts et Métiers (IBAM), 63p ;

2. Webographie

- ❖ Odoo. Progiciel de gestion intégré open source utilisée pour la gestion des ventes et de la facturation — consulté durant toutes les phase du projet.
https://www.odoo.com/fr_FR
- ❖ Laravel. Framework PHP open source utilisé pour le développement de l'application côté serveur — consulté plusieurs fois pendant le développement de la plateforme.
<https://laravel.com>
- ❖ React. Bibliothèque JavaScript utilisée pour la création d'interfaces utilisateur — consultée pour la conception des composants et l'utilisation des hooks.
<https://react.dev>
- ❖ GitHub. Plateforme de gestion de code source — utilisée pour le versionnage du code et la collaboration durant tout le projet.
<https://github.com>
- ❖ Stack Overflow. Forum en ligne — consulté à plusieurs reprises pour la résolution de problèmes techniques rencontrés pendant le développement.
<https://stackoverflow.com>
- ❖ YouTube. Plateforme de partage de vidéos — consultée pour des tutoriels sur Laravel, React, et la configuration de l'environnement de développement.
<https://www.youtube.com>
- ❖ Wikipedia – Two Tracks Unified Process (2TUP). Méthodologie en “Y” utilisée comme base pour l'analyse et la conception du système — consultée pour comprendre la démarche.
Consulté le 1er août 2025 —
https://fr.wikipedia.org/wiki/Two_Tracks_Unified_Process
- ❖ ProjectManagement.com – eXtreme Programming (XP). Méthode agile axée sur la qualité du code et l'adaptabilité — consultée pour la comparaison des approches agiles.
Consulté le 2 août 2025 —
<https://www.projectmanagement.com/wikis/609078/extreme-programming>
- ❖ Scrum Guides. Méthode agile basée sur des itérations courtes — consultée pour la compréhension du cadre Scrum.
Consulté le 2 août 2025 — <https://scrumguides.org/scrum-guide.html>

- ❖ OMG – Business Process Model and Notation (BPMN). Standard de modélisation de processus métiers — consulté pour la comparaison avec UML.
Consulté le 2 août 2025 — <https://www.omg.org/bpmn>
- ❖ Wikipedia – Model-Based Testing (MBT). Méthode de tests fondée sur la modélisation — consultée pour la comparaison avec UML.
Consulté le 2 août 2025 — https://fr.wikipedia.org/wiki/Model-based_testing
- ❖ SchoolMouv – Le SysML. Langage de modélisation pour l’ingénierie système — consulté pour la comparaison avec UML.
Consulté le 2 août 2025 — <https://www.schoolmouv.fr/cours/le-sysml/fiche-de-cours>
- ❖ OMG – Unified Modeling Language (UML). Langage de modélisation unifié — consulté pour la conception des diagrammes UML du système.
Consulté le 2 août 2025 — <https://www.omg.org/spec/UML>
- ❖ Zoho Books. Application de facturation en ligne pour PME — consultée pour l’étude comparative des outils ERP de facturation.
Consulté le 2 septembre 2025 — <https://www.zoho.com/books/>
- ❖ Xero. Logiciel de comptabilité cloud — consultée pour l’étude comparative des outils ERP de facturation.
Consulté le 2 septembre 2025 — <https://www.xero.com/>
- ❖ FreshBooks. Application de comptabilité et de facturation — consultée pour l’étude comparative des outils ERP de facturation.
Consulté le 2 septembre 2025 — <https://www.freshbooks.com/>
- ❖ Sage Intacct. Solution cloud de gestion financière — consultée pour l’étude comparative des outils ERP de facturation.
Consulté le 2 septembre 2025 — <https://www.sage.com/intacct>
- ❖ NetSuite ERP. Logiciel cloud complet pour la gestion d’entreprise — consulté pour l’étude comparative des outils ERP de facturation.
Consulté le 2 septembre 2025 — <https://www.netsuite.com/>

Table des matières

SOMMAIRE	i
DÉDICACE.....	ii
REMERCIEMENTS.....	iii
LISTE DES SIGLES ET ABREVIATIONS.....	iv
LISTE DES FIGURES ET GRAPHIQUES	vi
LISTE DES TABLEAUX	vii
INTRODUCTION GÉNÉRALE	1
CHAPITRE I : Présentation des structures de formations et d'accueil.....	2
I. Présentation de la structure de formation	2
1. Historique	2
2. Objectif.....	2
3. Organisation	2
a). L'organe statutaire.....	3
b). L'organe d'exécution	4
4. Filières de formation	4
a). Licences professionnelles.....	4
b). Masters	5
II. Présentation de la structure d'accueil.....	5
1. Identité.....	5
2. Historique	6
3. Mission et objectifs stratégiques	6
4. Domaines de compétence et prestations.....	6
5. Produits digitaux et partenaires	7
6. Organisation	7
CHAPITRE II : Analyse et la conception	9
I. L'étude préalable.....	9
1. Présentation du thème	9
a). Problématique.....	9
b). Objectifs	10
c). Résultats attendus.....	10
2. Méthodologie	10
a). Cycle de développement	11
b). Langage de modélisation.....	14
3. Présentation du groupe de travail	16
a). Groupe de pilotage	16

b).	Groupe des utilisateurs	16
c).	Groupe de projet.....	16
4.	Planning de travail.....	16
II.	Expression des besoins.....	17
1.	L'étude de l'existant.....	17
a).	Présentation du processus actuel	17
b).	Présentation du dispositif actuel.....	20
c).	Limites du dispositif actuel	20
2.	Exigences fonctionnelles.....	20
3.	Présentation du processus de fonctionnement cible.....	22
a).	Solutions envisageables.....	22
4.	Spécifications fonctionnelles.....	24
a).	Identification des acteurs.....	24
b).	Cas d'utilisation.....	24
c).	Diagramme des cas d'utilisation	26
d).	Description textuelle de certains cas d'utilisation.....	28
e).	Diagramme de séquences	31
5.	Spécifications techniques	35
a).	Mise à disposition des conditions de travail.....	35
b).	Architecture de développement.....	36
III.	Conception globale.....	37
1.	Diagramme des classes.....	37
a).	Dictionnaire de données	37
b).	Diagramme des classes.....	39
2.	Diagrammes d'état-transitions	41
3.	Diagramme de déploiement	43
IV.	Réalisation.....	43
1.	Présentation des outils développement	44
a).	Langages utilisés	44
b).	Frameworks de développement.....	44
c).	Outils de conception.....	46
2.	Infrastructure et l'environnement d'exécution.....	47
a).	Choix du serveur web.....	47
b).	Choix Système de Gestion de Base de données (SGBD).....	48
3.	Présentation de l'architecture logicielle	50
4.	Quelques interfaces (IHM) de notre application	52
5.	Tests	54

6. Mesures de sécurité	59
7. Estimation des coûts du projet	61
CHAPITRE III : Bilan du stage.....	66
I. Déroulement du stage et les activités menées	66
1. Activités menées	66
2. Connaissances acquises et compétences développées.....	67
II. Observations et suggestions	67
CONCLUSION GÉNÉRALE	68
BIBLIOGRAPHIE ET WEBOGRAPHIE	I
1. Bibliographie	I
2. Webographie	II
Table des matières	IV
ANNEXES	VII
I. ANNEXE 1 : Langage UML.....	VII
II. ANNEXE 2 : Le dictionnaires des données.	VII

ANNEXES

I. ANNEXE 1 : Langage UML

UML est l'acronyme anglais pour « Unified Modeling Language ». Il se traduit par « Langage de modélisation unifié ». C'est un langage de modélisation graphique à base de pictogrammes conçu pour fournir une méthode normalisée permettant de visualiser la conception d'un système. Ce langage est utilisé pour la spécification, la visualisation, la modification et la construction des documents nécessaires au bon développement d'un logiciel orienté objet. Il offre un standard de modélisation, pour représenter l'architecture logicielle. Grâce à UML, il est possible de générer tout ou une partie du code d'un logiciel à partir des divers documents réalisés. Ce langage de modélisation nous offre principalement 13 diagrammes (depuis sa deuxième version) pour modéliser un système. Ces diagrammes peuvent être utilisés selon la phase du développement d'un logiciel. En analyse, nous pouvons utiliser des :

- ❖ Diagrammes de cas d'utilisation : modélisent les besoins des utilisateurs ;
- ❖ Diagrammes de séquences vue de l'extérieur : présentent les scénarios entre les utilisateurs ;
- ❖ Diagrammes d'activités : c'est un enchaînement d'actions représentant un comportement du logiciel.
- ❖ En phase de conception, le développeur peut utiliser des :
 - ❖ Diagrammes de classes : pour représenter la structure interne du logiciel ;
 - ❖ Diagrammes d'objets : pour présenter l'état interne du logiciel à un instant donné ;
 - ❖ Diagrammes d'états-transitions : pour présenter l'évolution de l'état d'un objet ;
 - ❖ Diagrammes de séquence vue de l'intérieur : pour montrer les scénarios d'interactions avec les utilisateurs au sein du logiciel ;
 - ❖ Diagrammes de composants : pour présenter les composants physiques du logiciel ;
 - ❖ Diagrammes de déploiement : pour l'organisation matérielle du logiciel.

Ces diagrammes sont rarement tous implémentés dans le cadre du même projet. Le choix des diagrammes à mettre en œuvre dans la modélisation est généralement fonction de la nature du projet et de sa taille.

II. ANNEXE 2 : Le dictionnaires des données.

- **Classe : OdooConfiguration**

Tableau 26 : Dictionnaire de données de la classe « OdooConfiguration »

Attribut	Description	Type de donnée
id	Identifiant de la configuration Odoo	Entier
nomUtilisateur	Nom d'utilisateur Odoo (API)	Chaine de caractère
baseDeDonnee	Nom de la base Odoo	Chaine de caractère
url	URL de l'instance Odoo	Chaine de caractère
cleApi	Clé API qui permet de s'authentifier sur odoo	Chaine de caractère
estActif	Indique si la configuration est active	Booléen
derniereFactId	Dernière facture importée (réf.)	Chaine de caractère

- **Classe : Structure**

Tableau 27 : Dictionnaire de données de la classe « Structure »

Attribut	Description	Type de donnée
id	Identifiant de la structure	Entier
email	Adresse e-mail	Chaine de caractère
nom	Dénomination / raison sociale	Chaine de caractère
sigle	Sigle de la structure	Chaine de caractère
adresse	Adresse postale/physique	Chaine de caractère
estClient	Indique si la structure est un client	booléen

- **Classe : TypeStructure**

Tableau 28 : Dictionnaire de données de la classe « TypeStructure »

Attribut	Description	Type de donnée
id	Identifiant du type de structure	Entier
nom	Libellé du type de structure	Chaine de caractère

- **Classe : Domaine**

Tableau 29 : Dictionnaire de données de la classe « Domaine »

Attribut	Description	Type de donnée
id	Identifiant du domaine d'activité	Entier
nom	Libellé du domaine d'activité	Chaine de caractère

- **Classe : Contact**

Tableau 30 : Dictionnaire de données de la classe « Contact »

Attribut	Description	Type de donnée
id	Identifiant de la Contact	Entier
nom	Le nom du contact	Chaine de caractère
telephone1	Téléphone principal	Chaine de caractère
telephone1	Téléphone secondaire	Chaine de caractère

- **Classe : Compte**

Tableau 31 : Dictionnaire de données de la classe « Compte »

Attribut	Description	Type de donnée
id	Identifiant du Compte	Entier
numero_RIB	Le numero RIB du compte	Chaine de caractère
nomUtilisateur	Référence utilisateur propriétaire	Chaine de caractère

- **Classe : TypeCompte**

Tableau 32 : Dictionnaire de données de la classe « TypeCompte »

Attribut	Description	Type de donnée
id	Identifiant du type de compte	Entier

nom	Libellé du type de compte	Chaine de caractère
-----	---------------------------	---------------------

- **Classe : Facture**

Tableau 33 : Dictionnaire de données de la classe « Facture »

Attribut	Description	Type de donnée
id	Identifiant de la facture	Entier
reference	Référence ou numéro de la facture	Chaine de caractère
intitule	Intitulé ou l'objet de la facture	Chaine de caractère
numBonCommande	Référence du bon de commande	Chaine de caractère
statut	État de la facture	Chaine de caractère
tauxTVA	Taux de TVA appliqué	Numérique
montantHT	Montant hors taxes	Numérique
dateEcheance	Date limite de paiement	Date
dateEtablissement	Date d'établissement	Date
dateDepot	Date de dépôt chez le client ou la date de réception s'il s'agit d'une facture fournisseur.	Date
piceJointe	Chemin/réf. de la pièce jointe	Chaine de caractère

- **Classe : ReglementFacture**

Tableau 34 : Dictionnaire de données de la classe « ReglementFacture »

Attribut	Description	Type de donnée
id	Identifiant du règlement	Entier
datePaiement	Date effective du paiement	Date
dateEnregistrement	Date d'enregistrement système	Date
montant	Montant réglé	Numérique
pieceJointe	Url du Fichier Justificatif (reçu, virement...)	Chaine de caractère

- **Classe : ModePaiement**

Tableau 35 : Dictionnaire de données de la classe « ModePaiement »

Attribut	Description	Type de donnée
id	Identifiant du mode	Entier
libele	Libellé (Virement, Chèque, ...)	Chaine de caractère
description	Détails complémentaires	Chaine de caractère

- **Classe : Retenue**

Tableau 36 : Dictionnaire de données de la classe « Retenue »

Attribut	Description	Type de donnée
id	Identifiant de la retenue	Entier
motif	Motif de la retenue	Chaine de caractère
taux	Taux appliqué (%)	Numérique
retenueTVA	Retenue liée à la TVA	Booléen
pieceJointe	Url du fichier justificatif de retenue	Chaine de caractère

- **Classe : GarantieBonneExecution**

Tableau 37 : Dictionnaire de données de la classe « GarantieBonneExecution »

Attribut	Description	Type de donnée
id	Identifiant de la garantie	Entier
libele	Libellé de la garantie	Chaine de caractère
pourcentage	Pourcentage garanti (ex. 5%)	Numérique
dateDebut	Début de validité	Date
dateFin	Fin/échéance	Date
etatGarantie	État de la garantie	Chaine de caractère
pieceJointe	Copie/lettre de garantie	Chaine de caractère

- **Classe : Banque**

Tableau 38 : Dictionnaire de données de la classe « Banque »

Attribut	Description	Type de donnée
id	Identifiant de la banque	Entier
nom	Dénomination de la banque	Chaine de caractère

- **Classe : Utilisateur**

Tableau 39 : Dictionnaire de données de la classe « Utilisateur »

Attribut	Description	Type de donnée
id	Identifiant de l'utilisateur	Entier
nom	Nom complet	Chaine de caractère
email	Adresse e-mail	Chaine de caractère
hashMotDePasse	Mot de passe chiffré	Chaine de caractère
estActif	Statut du compte	Booléen
dateDerniereConnexion	Dernière connexion	Chaine de caractère
avatar	L'url de l'avatar (photo)	Chaine de caractère

- **Classe : Role**

Tableau 40 : Dictionnaire de données de la classe « Role »

Attribut	Description	Type de donnée
id	Identifiant du rôle	Entier
nom	Libellé du rôle	Chaine de caractère
description	Description du rôle	Chaine de caractère

- **Classe : Permission**

Tableau 41 : Dictionnaire de données de la classe « Permission »

Attribut	Description	Type de donnée
id	Identifiant de la permission	Entier
code	Code unique	Chaine de caractère
libele	Libellé lisible	Chaine de caractère
description	Description de la permission	Chaine de caractère